



**Facultatea de
Automatică și
Calculatoare**

Universitatea Politehnica Timișoara
Facultatea de Automatică și Calculatoare
Ingineria Sistemelor

LUCRARE DE DIPLOMĂ

Sistem Smart Home Local-First cu Detectia Anomaliilor Comportamentale prin Învățare Automată pe Edge

Absolvent
Vidac Denisa

Coordonator științific
Ș.l. dr. ing. Paul NEGÎRLA

Îndrumător
Asist. univ. drd. ing. Darius NEGÎRLA

Timișoara, 2026

Lucrarea prezintă proiectarea și implementarea unui sistem *smart home local-first*, care funcționează integral în rețeaua locală a locuinței, fără dependență de servicii cloud. Sistemul este organizat pe cinci straturi: senzori și actuatori, un nod ESP32 cu firmware Arduino, un [Raspberry Pi \(RPi\)](#) care găzduiește brokerul [Message Queuing Telemetry Transport \(MQTT\)](#) *Mosquitto*, o bază de date PostgreSQL/TimescaleDB, un API NestJS și o aplicație mobilă React Native. Nodul interior achiziționează temperatura și umiditatea (două senzore [Digital Humidity and Temperature \(DHT\)](#)¹¹), nivelul de gaz (MQ-2) și luminozitatea (două [Light-Dependent Resistor \(LDR\)](#)), și comandă trei [Light-Emitting Diode \(LED\)](#)-uri, un ventilator, un servomotor și un buzzer; o cameră ESP32-CAM cu senzor [Passive Infrared \(PIR\)](#) acoperă exteriorul. Contribuția principală o reprezintă un modul de **detectie de anomalii comportamentale** bazat pe algoritmul [Isolation Forest \(IF\)](#) (*Isolation Forest*), antrenat și executat **local**, pe Raspberry Pi, care învață tiparele normale ale locuinței și semnalează abaterile fără reguli prestabilite. Lucrarea descrie arhitectura, implementarea prototipului și metodologia de evaluare, evidențiind avantajele de confidențialitate, cost și autonomie ale abordării locale.

Cuvinte-cheie: smart home, Internet of Things, local-first, detectie de anomalii, Isolation Forest, edge computing, ESP32, MQTT, Raspberry Pi.

This thesis presents the design and implementation of a *local-first* smart home system that operates entirely within the home's local network, with no dependency on cloud services. The system is organized in five layers: sensors and actuators, an ESP32 node running Arduino firmware, a Raspberry Pi hosting the *Mosquitto* MQTT broker, a PostgreSQL/TimescaleDB database, a NestJS API and a React Native mobile application. The indoor node acquires temperature and humidity (two DHT11 sensors), gas level (MQ-2) and illumination (two LDRs), and drives three LEDs, a fan, a servo motor and a buzzer; an ESP32-CAM with a PIR sensor covers the outdoor area. The main contribution is a **behavioural anomaly detection** module based on the IF (*Isolation Forest*) algorithm, trained and executed **locally** on the Raspberry Pi, which learns the household's normal patterns and flags deviations without predefined rules. The thesis describes the architecture, the prototype implementation and the evaluation methodology, highlighting the privacy, cost and autonomy benefits of the local approach.

Keywords: smart home, Internet of Things, local-first, anomaly detection, Isolation Forest, edge computing, ESP32, MQTT, Raspberry Pi.

Cuprins

Listă de figuri	viii
Listă de tabele	ix
Glosar	x
1 Introducere	1
1.1 Context și motivație	1
1.2 Problema abordată și abordarea local-first	2
1.3 Obiectivele lucrării	3
1.4 Contribuții originale	4
1.5 Structura lucrării	5
2 Studii anterioare	6
2.1 Platforme smart home comerciale și open-source	6
2.1.1 Platforme comerciale	6
2.1.2 Platforme open-source	7
2.2 Protocoale de comunicație IoT	7
2.2.1 Comparatie MQTT / CoAP / HTTP	7
2.2.2 Justificarea alegerii MQTT	8
2.3 Edge computing și procesarea locală	9
2.4 Detecția de anomalii comportamentale	9
2.4.1 Taxonomia metodelor de detecție	9
2.4.2 Principiul izolării — ilustrare conceptuală	10
2.4.3 Justificarea alegerii Isolation Forest pentru sistem	11
2.5 Poziționarea lucrării față de stadiul actual	12
3 Metodologie și arhitectura sistemului	13
3.1 Principii de proiectare	13
3.2 Arhitectura de ansamblu	14
3.3 Cerințe funcționale și nefuncționale	15
3.3.1 Cerințe funcționale	15
3.3.2 Cerințe nefuncționale	17

3.4	Fluxul de date și protocolul MQTT	17
3.4.1	Topicuri MQTT și contractul de date	18
3.4.2	Last Will and Testament și QoS	19
3.4.3	Descoperire prin mDNS și provisioning Wi-Fi	19
3.5	Modelul de date	19
3.6	Alegeri tehnologice justificate	21
3.7	Pipeline-ul de detectare a anomaliilor — contribuția originală	23
3.7.1	Algoritmul Isolation Forest	23
3.7.2	Ingineria trăsăturilor (<code>features.py</code>)	24
3.7.3	Normalizare	25
3.7.4	Antrenarea modelului (<code>train.py</code>)	25
3.7.5	Inferența în timp real (<code>infer.py</code>)	25
3.7.6	Mecanismul de cold-start	26
3.8	Considerații de securitate	26
4	Implementarea prototipului	28
4.1	Firmware-ul nodului interior (ESP32 WROOM-32)	28
4.1.1	Pinout și configurare hardware	28
4.1.2	Citirea senzorilor	30
4.1.3	Actuatori	30
4.1.4	Comunicație MQTT, provisioning Wi-Fi și mDNS	31
4.2	Firmware-ul ESP32-CAM (nod exterior)	31
4.2.1	Serverul HTTP și stream-ul MJPEG	32
4.3	Gateway Raspberry Pi	32
4.4	API NestJS	32
4.4.1	Structura modulelor	33
4.4.2	Subscriber MQTT	33
4.4.3	WebSocket (Socket.IO)	34
4.4.4	Endpointuri REST și autentificare JWT	34
4.4.5	Modulul simulator	34
4.5	Motorul de detecție a anomaliilor cu Isolation Forest	35
4.5.1	Ingineria trăsăturilor (<code>features.py</code>)	35
4.5.2	Antrenarea modelului (<code>train.py</code>)	36
4.5.3	Inferența în timp real (<code>infer.py</code>)	36
4.6	Fluxuri principale — diagrame de secvență	37
4.6.1	Fluxul de monitorizare	37
4.6.2	Fluxul de comandă a actuatorilor	37
4.6.3	Fluxul de detecție a anomaliilor	38
4.7	Aplicația mobilă	39
4.7.1	Discovery-ul gateway-ului	39
4.7.2	Recepția datelor în timp real (Socket.IO)	39

4.7.3	Trimiterea comenzilor către actuatori	40
4.7.4	Ecranele aplicației	41
4.8	Sumar	42
5	Rezultate și evaluare	43
5.1	Mediul și metodologia de testare	43
5.1.1	Configurația hardware de testare	43
5.1.2	Modul simulator	44
5.1.3	Piramida de testare propusă	44
5.2	Verificarea funcțională a sistemului	45
5.2.1	Fluxul de comandă	47
5.2.2	Fluxul de alertă de gaz	47
5.3	Performanță operațională	47
5.3.1	Latența comandă-actuator	47
5.3.2	Frecvența de publicare senzori	47
5.4	Evaluarea detecției de anomalii	49
5.4.1	Protocolul de evaluare propus	49
5.4.2	Parametrii actuali ai modelului	50
5.4.3	Mecanism de cold-start	50
5.4.4	Rezultate ilustrative	50
5.5	Discuție: avantaje confirmate și limitări actuale	51
5.5.1	Avantaje confirmate ale abordării	51
5.5.2	Limitări actuale	52
5.6	Sinteză	52
6	Concluzii și dezvoltări viitoare	54
6.1	Concluzii generale	54
6.2	Contribuții	55
6.3	Limitări ale implementării curente	55
6.4	Direcții de dezvoltare viitoare	56
6.5	Remarci finale	57
	Bibliografie	58

Listă de figuri

2.1	Principiul izolării: un punct anomal este separat prin puține partiționări aleatoare, în timp ce un punct normal (din zona densă) necesită multe partiții.	11
3.1	Arhitectura de ansamblu a sistemului, organizată pe cinci straturi. Toate componentele operează exclusiv în rețeaua locală, fără dependență de cloud.	14
3.2	Diagrama use-case a sistemului smart home. Actorul principal este utilizatorul, iar motorul ML participă ca actor secundar (sistem) prin generarea automată de alerte de anomalie.	16
3.3	Fluxul de date în sistem. Datele de senzori circulă de la ESP32 prin Mosquitto spre NestJS, care le persistă în TimescaleDB, le transmite aplicației prin WebSocket și le evaluează prin motorul ML. Comenzile urmează traseul invers (linie punctată).	17
3.4	Diagrama entitate-relație a bazei de date. Tabelul <code>sensor_readings</code> este hypertable TimescaleDB partitionat automat pe coloana <code>time</code>	20
3.5	Pipeline-ul complet al motorului ML: de la colectarea datelor din TimescaleDB, prin ingineria trăsăturilor și normalizare, până la antrenarea periodică a modelului Isolation Forest și inferența în timp real. Bucla de reantrenare zilnică și mecanismul de cold-start sunt evidențiate.	23
4.1	Schema de conexiuni a nodului interior ESP32 WROOM-32 (pinout conform Tabelul 4.1).	29
4.2	Diagrama de secvență pentru fluxul de monitorizare: ESP32 → Mosquitto → NestJS → TimescaleDB → App.	37
4.3	Diagrama de secvență pentru fluxul de comandă a actuatorilor: App → NestJS → Mosquitto → ESP32 → actuator fizic.	38
4.4	Diagrama de secvență pentru detecția anomaliei comportamentale cu IF: NestJS apelează <code>infer.py</code> și generează alertă <code>ML_ANOMALY</code>	38
4.5	Capturi reale din aplicația mobilă: (a) Dashboard — scor de siguranță, medii ambientale și tile-uri pe camere; (b) Alerte — cronologie cu severitate și stare de confirmare; (c)–(d) Control — cameră live, scene, draperii (servo), LED-uri și ventilator, cu indicator „nod offline” când nodul nu este conectat.	42
	(a) Dashboard	42

(b)	Alerte	42
(c)	Control (sus)	42
(d)	Control (jos)	42
5.1	Piramida de testare propusă pentru sistem: teste unitare la bază, teste de integrare la nivel intermediar și teste end-to-end la vârf. Lateral — testarea manuală pe hardware.	44
5.2	Exemplu ilustrativ al scorului de anomalie pe 24 h (date sintetice / preliminare): serie temporală a scorului IF, prag la -0.1 și puncte detectate ca anomalii (roșu). Notă: datele sunt ilustrative și nu provin din operare reală de durată.	51

Listă de tabele

2.1	Comparație MQTT, CoAP și HTTP pentru aplicații IoT	8
3.1	Topicurile MQTT ale sistemului smart home.	18
3.2	Alegeri tehnologice — cerință, soluție adoptată și justificare.	22
3.3	Cele 12 trăsături ale modelului Isolation Forest, în ordinea fixă din FEA-TURE_NAMES	24
4.1	Pinout complet al nodului interior (ESP32 WROOM-32).	29
4.2	Principalele endpointuri Representational State Transfer (REST) ale Application Programming Interface (API)-ului NestJS.	34
4.3	Cele 12 trăsături utilizate de motorul Isolation Forest.	35
5.1	Verificarea funcțională a componentelor sistemului.	46
5.2	Indicatori de performanță operațională — valori preliminare / ilustrative	48

Glosar

ADC Analog-to-Digital Converter. [14](#), [20](#), [22](#), [28](#), [30](#), [47](#), [52](#)

API Application Programming Interface. [ix](#), [3](#), [4](#), [5](#), [7](#), [11](#), [15](#), [22](#), [26](#), [28](#), [32](#), [34](#), [39](#), [42](#), [43](#), [44](#), [46](#), [47](#), [48](#), [54](#), [57](#)

AUC Area Under the Curve. [49](#)

CPU Central Processing Unit. [48](#)

DHT Digital Humidity and Temperature. [ii](#), [iii](#), [3](#), [29](#), [30](#), [31](#), [41](#), [43](#), [47](#), [54](#), [55](#)

edge paradigmă de procesare a datelor la marginea rețelei, în apropierea sursei de date. [55](#)

ESP32 Espressif Systems 32-bit SoC. [28](#)

GDPR General Data Protection Regulation. [1](#), [5](#), [13](#), [27](#), [51](#), [55](#)

GPIO General-Purpose Input/Output. [14](#), [30](#)

IF Isolation Forest. [ii](#), [iii](#), [vii](#), [viii](#), [2](#), [3](#), [4](#), [5](#), [28](#), [38](#), [43](#), [48](#), [49](#), [50](#), [51](#), [52](#), [53](#)

IoT Internet of Things. [1](#), [3](#), [5](#), [6](#), [7](#), [9](#), [10](#), [13](#), [22](#), [51](#), [52](#), [54](#), [55](#), [56](#), [57](#)

IP Internet Protocol. [54](#)

JSON JavaScript Object Notation. [40](#)

JWT JSON Web Token. [3](#), [5](#), [15](#), [22](#), [26](#), [34](#), [39](#), [40](#), [42](#), [46](#)

LAN Local Area Network. [2](#), [3](#), [5](#), [13](#), [27](#), [47](#), [48](#), [51](#), [56](#)

LDR Light-Dependent Resistor. [ii](#), [iii](#), [3](#), [14](#), [20](#), [28](#), [29](#), [41](#), [43](#), [47](#), [48](#), [52](#), [54](#)

LED Light-Emitting Diode. [ii](#), [iii](#), [vii](#), [3](#), [14](#), [29](#), [31](#), [40](#), [41](#), [42](#), [43](#), [54](#)

mDNS Multicast Domain Name System. [3](#), [5](#), [14](#), [15](#), [19](#), [43](#), [54](#)

MJPEG Motion JPEG. [46](#), [48](#), [54](#)

ML Machine Learning. [2](#), [3](#), [4](#), [5](#), [7](#), [9](#), [11](#), [12](#), [13](#), [15](#), [16](#), [26](#), [35](#), [43](#), [46](#), [48](#), [51](#), [52](#), [53](#), [54](#), [56](#), [57](#)

MQTT Message Queuing Telemetry Transport. [ii](#), [iii](#), [3](#), [4](#), [5](#), [7](#), [8](#), [9](#), [13](#), [14](#), [15](#), [16](#), [17](#), [18](#), [19](#), [27](#), [31](#), [32](#), [44](#), [45](#), [46](#), [47](#), [52](#), [53](#), [54](#), [56](#), [57](#)

NVS Non-Volatile Storage. [5](#)

OASIS Organization for the Advancement of Structured Information Standards. [17](#)

PIR Passive Infrared. [ii](#), [iii](#), [3](#), [31](#), [35](#), [42](#), [43](#), [54](#)

PWM Pulse-Width Modulation. [14](#), [29](#), [30](#)

RAM Random-Access Memory. [48](#)

REST Representational State Transfer. [ix](#), [3](#), [15](#), [22](#), [26](#), [34](#), [39](#), [40](#), [42](#)

ROC Receiver Operating Characteristic. [49](#)

RPi Raspberry Pi. [ii](#), [2](#), [3](#), [4](#), [5](#), [48](#), [51](#), [53](#), [54](#), [55](#)

SVM Support Vector Machine. [10](#)

TLS Transport Layer Security. [52](#), [53](#), [56](#)

UI User Interface. [41](#)

Wi-Fi Wireless Fidelity. [3](#), [28](#), [42](#), [47](#)

1 Introducere

1.1 Context și motivație

Locuințele inteligente reprezintă una dintre aplicațiile cu cea mai rapidă expansiune din domeniul [Internet of Things \(IoT\)](#). Dacă la începutul deceniului trecut termenul „smart home” evoca un lux rezervat câtorva entuziaști, astăzi milioane de gospodării integrează senzori de temperatură, camere de supraveghere, sisteme de iluminat și asistenți vocali conectați la internet. Piața globală a locuințelor inteligente a depășit \$80 de miliarde în 2023 și se preconizează o creștere susținută în anii următori [34, 17]. La baza acestei transformări stă tocmai convergența [IoT](#): dispozitive cu resurse limitate care colectează date din mediul fizic, le transmit în rețea și declanșează acțiuni automatizate în funcție de starea mediului [4].

Cu toate acestea, marea majoritate a soluțiilor comerciale disponibile astăzi (Google Home, Amazon Alexa, Apple HomeKit, Tuya etc.) se bazează pe un model *cloud-first*: datele brute ale senzorilor sunt trimise pe servere aflate în altă țară, procesate acolo și returnate aplicației mobile printr-un lanț de apeluri de rețea. Această arhitectură ridică o serie de probleme grave:

- **Confidențialitate și protecția datelor.** Informații sensibile privind comportamentul cotidian al locatarilor (ore de prezență acasă, temperaturi, mișcări în cameră) sunt stocate pe infrastructuri terțe, adesea în afara jurisdicției [General Data Protection Regulation \(GDPR\)](#) [41, 13]. Chiar atunci când furnizorii respectă legislația europeană, utilizatorul nu deține controlul efectiv asupra datelor generate în propria locuință [2].
- **Dependența de conectivitatea la internet.** Dacă furnizorul de servicii întrerupe suportul pentru un produs sau dacă conexiunea la internet este indisponibilă temporar, întregul sistem poate deveni nefuncțional. Experiențele legate de retragerea prematură a unor platforme (Google Stadia, SmartThings Hub v2 etc.) au demonstrat că *vendor lock-in*-ul reprezintă un risc real pentru utilizatorii care investesc în ecosisteme proprietare [21].

- **Cost și complexitate.** Abonamentele lunare pentru platformele cloud, costul gateway-urilor proprietare și imposibilitatea de a integra hardware eterogen cresc semnificativ bariera de acces. Soluțiile open-source existente (Home Assistant, OpenHAB) oferă alternativă, dar cer cunoștințe tehnice avansate și nu includ componente de analiză inteligentă a comportamentului.
- **Latență și procesare la margine rețelei.** Trimiterea fiecărei citiri de senzor spre cloud și așteptarea unui răspuns introduce latențe inacceptabile pentru scenarii de siguranță (detectare gaz, alertă de intruziune nocturnă). Paradigma *edge computing* propune procesarea locală a datelor, acolo unde sunt generate [37].

Aceste limitări nu sunt de natură exclusiv tehnică: ele reflectă un dezechilibru structural în care controlul locuinței proprii este delegat unor entități externe. Lucrarea de față propune o arhitectură alternativă, *local-first*, care rezolvă fiecare dintre problemele enumerate anterior.

1.2 Problema abordată și abordarea local-first

Problema centrală abordată în această lucrare este următoarea: *Este posibil să se construiască un sistem de locuință inteligentă complet funcțional, capabil de monitorizare în timp real, control al actuatorilor, supraveghere video și detecție inteligentă a anomaliilor comportamentale, folosind exclusiv infrastructură locală, fără nicio dependență de servicii cloud, și cu un cost total de hardware accesibil?*

Răspunsul propus este o arhitectură **local-first** [21]: toate datele rămân în rețeaua locală (**Local Area Network (LAN)**) a locatarului, procesarea se realizează pe un gateway local (**RPi**), iar aplicația mobilă comunică direct cu serverul intern fără a tranzita internetul. Această abordare se înscrie în tendința mai largă de *edge computing* [37, 32]: în loc să se centralizeze inteligența în cloud, logica de decizie este mutată cât mai aproape de sursele de date (senzori, camere, actuatori).

Sistemul implementat funcționează integral în **LAN**, inclusiv atunci când conexiunea la internet este indisponibilă. Singurul trafic care poate opțional ieși din rețeaua locală este notificarea push, iar și aceasta poate fi dezactivată. Datele personale ale utilizatorului (temperaturi, ore de prezență, imagini de la cameră) nu părăsesc niciodată locuința fără consimțământ explicit.

Componenta de **Machine Learning (ML)** — motorul de detecție a anomaliilor comportamentale bazat pe algoritmul **IF** — rulează direct pe **RPi**, eliminând necesitatea unui serviciu cloud de inferență. Aceasta este contribuția originală centrală a lucrării, detaliată în Secțiunea 1.4 și dezvoltată extensiv în capitolele Capitolul 3 și Capitolul 4.

1.3 Obiectivele lucrării

Lucrarea își propune proiectarea, implementarea și evaluarea unui sistem de locuință inteligentă local-first, cu scopuri concrete structurate pe cinci straturi arhitecturale. Obiectivele principale sunt:

1. **Sistem funcțional pe cinci straturi.** Proiectarea unei arhitecturi coerente care integrează: senzori și actuatori fizici, noduri **IoT** bazate pe microcontrolere ESP32 programate în Arduino C++, un gateway local pe **RPi**, un **API REST** + WebSocket și o aplicație mobilă — toate comunicând în **LAN** fără cloud.
2. **Achiziție de date cu hardware real.** Conectarea și programarea celor 11 componente hardware reale ale nodului interior (ESP32 WROOM-32 cu două senzore **DHT11**, un senzor de gaz MQ-2, două rezistențe fotosensibile **LDR**, trei **LED**-uri, un buzzer, un servomotor SG90 și un ventilator 5V) și a nodului exterior (ESP32-CAM cu senzor **PIR** și cameră OV2640).
3. **Control al actuatorilor în timp real.** Implementarea unui protocol de comandă bidirecțional prin **MQTT** care permite aprinderea/stingerea **LED**-urilor, pornirea ventilatorului, mișcarea servo-ului (draperii), activarea buzzerului și configurarea **Wireless Fidelity (Wi-Fi)** — toate inițiate din aplicația mobilă cu latență sub o secundă în **LAN**.
4. **Motor de ML local pentru detecția anomaliilor.** Implementarea unui pipeline complet de **IF** (antrenare, normalizare, inferență în timp real, reantrenare automată zilnică) care rulează exclusiv pe **RPi**, fără apeluri externe, și care generează alerte de tip **ML_ANOMALY** integrate în sistemul de notificări al aplicației mobile.
5. **Aplicație mobilă cross-platform.** Dezvoltarea unei aplicații React Native / Expo cu patru tab-uri funcționale (Dashboard cu date live, Istoric cu grafice, Alerte, Control actuatori + cameră), care se conectează automat la gateway prin **Multicast Domain Name System (mDNS)** sau adresă IP și oferă autentificare **JSON Web Token (JWT)**.
6. **Independență completă de cloud.** Demonstrarea că întregul sistem (monitorizare, control, **ML**, video) funcționează fără nicio dependență de servicii externe, asigurând confidențialitatea datelor și reziliența la lipsa conectivității la internet.
7. **Cost redus și reproductibilitate.** Construirea sistemului din componente disponibile comercial, cu un cost total de hardware accesibil, și documentarea completă a firmware-ului, configurației serverului și structurii bazei de date pentru a permite reproducerea independentă.

1.4 Contribuții originale

Principala contribuție originală a lucrării este **implementarea unui motor de detecție a anomaliilor comportamentale bazat pe algoritmul IF [24], rulat în întregime local pe RPi.**

Algoritmul IF, propus de Liu, Ting și Zhou [24], izolează punctele anormale printr-un mecanism de partiționare aleatoare a spațiului de caracteristici: punctele rare (anormale) sunt separate prin mai puțini pași față de punctele normale, rezultând o *lungime de cale medie* mai mică și, implicit, un scor de anomalie mai mare. Această proprietate îl face deosebit de potrivit pentru scenarii de tip *edge*: nu necesită etichete de antrenare (algoritm nesupervizat), are complexitate computațională redusă și poate fi reantrenant periodic pe date noi fără intervenție umană.

În contextul acestui sistem, motorul ML lucrează cu 12 trăsături extrase din citirile senzorilor (temperaturi de la ambele senzore DHT11, umiditate, nivel de gaz normalizat, intensitate luminoasă normalizată logaritmice, prezența mișcării, ora zilei, ziua săptămânii, indicator de weekend/noapte, variația temperaturii pe o oră și numărul de evenimente de mișcare în 10 minute). Modelul este antrenat pe ultimele 30 de zile de date și reantrenant automat zilnic la ora 03:00 printr-un serviciu systemd. Inferența se realizează în timp real la fiecare citire nouă de senzori, fără latența unui apel de rețea spre cloud.

Această abordare reprezintă un avans față de soluțiile comerciale existente, care fie nu includ deloc componente de ML, fie externalizează procesarea spre servicii cloud cu implicații de confidențialitate. Spre deosebire de metodele supervizate care necesită date etichetate (greu de obținut în medii rezidențiale), IF este *nesupervizat* și se adaptează automat la rutina specifică a locatarilor [7, 15, 9].

Contribuțiile secundare ale lucrării includ:

- **Integrarea completă firmware broker API aplicație mobilă.** Lanțul complet de la firmware Arduino C++ pe ESP32, prin brokerul Mosquitto MQTT, prin API-ul NestJS cu TypeORM și TimescaleDB, până la interfața React Native — implementat și funcțional pe hardware fizic.
- **Arhitectura de date time-series.** Utilizarea TimescaleDB (extensie PostgreSQL orientată pe serii de timp) pentru stocarea eficientă a citirilor de senzori, cu indexare automată pe timestamp și suport pentru interogări de agregare pe intervale temporale.
- **Provisioning Wi-Fi și descoperire automată în rețea.** Mecanism de configurare inițială prin hotspot propriu al nodului ESP32 (SmartHome-Setup), cu stocare

a credențialelor în [Non-Volatile Storage \(NVS\)](#) și descoperire automată a gateway-ului prin [mDNS](#) (`smarthome.local`).

- **Modul de simulare pentru dezvoltare fără hardware.** Un modul `simulator/` integrat în backend-ul NestJS care generează date realiste ale senzorilor (inclusiv variabilele extrase ulterior de motorul [ML](#)), permițând dezvoltarea și testarea completă a stivei software fără acces la dispozitivele fizice.

1.5 Structura lucrării

Lucrarea este organizată în șase capitole, fiecare abordând un aspect distinct al sistemului:

Capitolul 1 — Introducere (capitolul de față) prezintă contextul domeniului, motivațiile care au condus la abordarea local-first, obiectivele concrete ale sistemului și contribuțiile originale.

Capitolul 2 — Studiu bibliografic trece în revistă literatura relevantă: evoluția sistemelor [IoT](#) și a locuințelor inteligente, protocoale de comunicație cu accent pe [MQTT](#), tehnici de detecție a anomaliilor în date de senzori (comparație între [IF](#), LOF, One-Class SVM și metode bazate pe rețele neuronale), paradigma *edge computing* și cadrul legal [GDPR](#) privind datele personale în medii rezidențiale.

Capitolul 3 — Metodologie și proiectare descrie arhitectura pe cinci straturi a sistemului, deciziile de proiectare (alegerea componentelor hardware, a stivei software, a bazei de date time-series), modelarea entităților, fluxurile de date [MQTT](#) și pipeline-ul complet al motorului [ML](#).

Capitolul 4 — Implementare detaliază realizarea efectivă a fiecărui strat: firmware-ul Arduino C++ al nodului interior și al ESP32-CAM, configurarea brokerului Mosquitto și a [RPi](#) ca gateway, implementarea [API](#)-ului NestJS cu modulele de [MQTT](#), WebSocket, autentificare [JWT](#) și [ML](#), și dezvoltarea aplicației mobile React Native cu toate funcționalitățile sale.

Capitolul 5 — Rezultate și evaluare prezintă rezultatele obținute: funcționarea sistemului pe hardware real, evaluarea motorului de [ML](#) pe date ilustrative (cu date preliminare marcate explicit ca atare), latențele măsurate în [LAN](#) și o discuție critică a limitărilor actuale și a direcțiilor de îmbunătățire.

Capitolul 6 — Concluzii sintetizează contribuțiile lucrării, evaluează măsura în care obiectivele au fost atinse și propune direcții de cercetare și dezvoltare ulterioară, inclusiv extinderea la mai multe noduri, integrarea unor senzori suplimentari și posibilitatea unui model [ML](#) mai sofisticat (autoencodere, LSTM) pentru detecția anomaliilor.

2 Studii anterioare

Prezentul capitol trece în revistă stadiul actual al domeniilor relevante pentru această lucrare: platformele de automatizare rezidențială, protocoalele de comunicație **IoT**, paradigmele de calcul la margine (edge/fog) și metodele de detecție a anomaliilor comportamentale. Pe baza acestei analize, sunt identificate lacunele din literatura de specialitate pe care sistemul propus urmărește să le acopere.

2.1 Platforme smart home comerciale și open-source

Piața automatizărilor rezidențiale a cunoscut o expansiune rapidă în ultimul deceniu, atât în segmentul comercial cât și în cel open-source. Stojkoska și Trivodaliev [34] oferă o trecere în revistă cuprinzătoare a provocărilor și soluțiilor din domeniu, evidențiind că interoperabilitatea, securitatea și confidențialitatea datelor rămân problemele centrale nerezolvate pe deplin de nicio platformă.

2.1.1 Platforme comerciale

Printre soluțiile comerciale cu cea mai largă adopție se numără Amazon Alexa/Echo, Google Home și Apple HomeKit. Toate trei se bazează pe arhitecturi hibride cloud-dispozitiv: comenzile utilizatorului sunt transmise serverelor producătorului, procesate în cloud și returnate dispozitivului. Această abordare oferă avantaje clare — actualizări automate, recunoaștere vocală de înaltă calitate, ecosisteme bogate de aplicații — dar implică o serie de dezavantaje fundamentale. Alam et al. [1] subliniază că dependența de conectivitate la internet face ca astfel de sisteme să devină nefuncționale în absența conexiunii sau la întreruperea serviciului cloud al producătorului.

Aspectul confidențialității este tratat în detaliu de Zheng et al. [41], care arată că datele colectate de dispozitivele smart home comerciale — inclusiv obiceiuri zilnice, prezența la domiciliu sau conversații ambientale — sunt transferate și stocate pe servere terțe, deseori în jurisdicții cu standarde de protecție a datelor diferite. O analiză de securitate sistematică realizată de Alrawi et al. [2] pe un eșantion larg de dispozitive

IoT de consum a identificat vulnerabilități la nivelul firmware-ului, al protocoalelor de comunicație și al API-urilor cloud, concluzionând că nivelul de securitate al ecosistemelor comerciale rămâne nesatisfăcător în raport cu amenințările actuale.

2.1.2 Platforme open-source

La polul opus se situează platformele open-source, dintre care Home Assistant și openHAB sunt cele mai reprezentative. Home Assistant, scris în Python și distribuit sub licență Apache 2.0, rulează local pe hardware modest (inclusiv Raspberry Pi) și oferă o interfață web completă, suport pentru sute de integrări și un motor de automatizare bazat pe evenimente. openHAB, scris în Java, adoptă o filosofie similară de independență față de cloud, cu accent pe extensibilitate prin plugin-uri.

Avantajul esențial al acestor platforme este că întreaga logică de control și toate datele rămân în rețeaua locală a utilizatorului. Totuși, curba de învățare este semnificativă, iar suportul pentru ML nativ (mai ales pentru detecția de anomalii) este limitat sau realizat prin integrări externe.

Sistemul propus în această lucrare împărtășește filosofia local-first a platformelor open-source, dar se distinge prin integrarea unui motor de ML propriu — Isolation Forest — rulat direct pe gateway-ul Raspberry Pi, fără a depinde de nicio bibliotecă externă de automatizare generică.

2.2 Protocoale de comunicație IoT

Alegerea protocolului de comunicație are implicații directe asupra consumului de resurse, a latenței, a scalabilității și a fiabilității unui sistem IoT. Cele mai utilizate protocoale în domeniu sunt MQTT, CoAP și HTTP. Naik [29] realizează o comparație detaliată a acestora, evidențiind că nu există o soluție universală optimă, alegerea depinzând de constrângerile concrete ale aplicației.

2.2.1 Comparație MQTT / CoAP / HTTP

Tabelul 2.1 sintetizează principalele caracteristici ale celor trei protocoale din perspectiva aplicabilității IoT.

Tabelul 2.1: Comparație MQTT, CoAP și HTTP pentru aplicații IoT

Caracteristică	MQTT	CoAP	HTTP/1.1
Transport	TCP	UDP	TCP
Model comunicație	Publish/Subscribe (broker central)	Request/Response (P2P sau proxy)	Request/Response (client-server)
Overhead antet	Minim (2–5 B antet fix)	Redus (4 B antet fix)	Mare (text, zeci-sute de octeți)
QoS	3 niveluri (0, 1, 2)	2 tipuri (confirmabil / neconfirmabil)	Implicit (TCP garantează livrarea)
Last Will / Retained	Da (suport nativ)	Nu (necesită extensii)	Nu
Potrivire IoT	Excelentă (multe-lamulți, noduri cu resurse limitate)	Bună (rețele cu pierderi, IPv6/6LoWPAN)	Moderată (overhead ridicat, nepotrivit pentru microcontrole)
Securitate	TLS/SSL opțional	DTLS opțional	TLS/SSL standard
Standard	OASIS [30]	IETF RFC 7252 [36]	IETF RFC 7230

2.2.2 Justificarea alegerii MQTT

MQTT a fost conceput explicit pentru conexiuni cu lățime de bandă mică și dispozitive cu resurse limitate [30]. Modelul publish/subscribe decuplează producătorul de date (nodul ESP32) de consumatori (backend NestJS, motor ML), permitând mai multor abonați să recepționeze simultan aceleași date fără ca nodul să cunoască numărul sau identitatea acestora.

Caracteristica *Last Will and Testament* este esențială pentru monitorizarea stării nodurilor: la conectare, fiecare nod înregistrează un mesaj de „ultimă voință” pe topicul `smarthome/{nodeId}/status`; dacă conexiunea se întrerupe neașteptat, brokerul publică automat acest mesaj, informând restul sistemului. Mesajele *retained* asigură că un abonat nou primește imediat ultima valoare a unui topic, fără a aștepta o nouă publicare.

Mosquitto, implementarea de referință pentru medii embedded [22], rulează eficient pe Raspberry Pi 3B+ și suportă autentificare prin nume de utilizator și parolă, acoperind cerințele de securitate minimale ale rețelei locale. Combinația MQTT + Mosquitto este larg utilizată în proiecte IoT de cercetare [29], ceea ce facilitează reproductibilitatea experimentelor.

CoAP ar fi o alternativă pentru rețele cu pierderi mari sau IPv6/6LoWPAN, scenarii irelevante pentru o rețea Wi-Fi locală. HTTP, deși simplu de implementat, ar impune un overhead de zeci de ori mai mare față de MQTT pentru aceeași sarcină utilă de date senzoriale, epuizând rapid memoria stack-ului TCP de pe microcontroler.

2.3 Edge computing și procesarea locală

Paradigma *edge computing* plasează capacitatea de calcul cât mai aproape de sursa de date, reducând latența, consumul de lățime de bandă și dependența de conectivitate la internet. Shi et al. [37] definesc edge computing ca extensie a modelului cloud la marginea rețelei și argumentează că este esențial pentru aplicații cu cerințe stricte de latență și confidențialitate.

În contextul IoT, Preamsankar et al. [32] demonstrează că procesarea locală reduce semnificativ volumul de date transmis spre cloud și permite reacții în timp real imposibil de obținut printr-o arhitectură cu cloud central. Lin et al. [23] extind analiza la scenarii de calcul mobil, arătând că un nod edge (cum ar fi un Raspberry Pi) poate prelua sarcini de inferență ML de complexitate medie cu latențe sub o secundă.

Sistemul proiectat în această lucrare adoptă integral filozofia edge computing: Raspberry Pi 3B+ acționează ca nod edge care stochează toate datele locale (PostgreSQL + TimescaleDB), execută inferența Isolation Forest în timp real și publică alerte fără a implica niciun serviciu extern. Această arhitectură garantează funcționarea completă a sistemului chiar și în absența totală a internetului, răspunzând direct preocupărilor de autonomie și confidențialitate identificate în literatura de specialitate.

2.4 Detecția de anomalii comportamentale

Detecția de anomalii reprezintă procesul de identificare a observațiilor care se abat semnificativ de la comportamentul așteptat al unui sistem. Chandola et al. [7] oferă o taxonomie exhaustivă a metodelor existente, grupate în funcție de tipul de supraveghere și de tehnica statistică utilizată.

2.4.1 Taxonomia metodelor de detecție

Metode statistice. Presupun că datele normale urmează o distribuție probabilistică parametrică (Gaussiană, Poisson etc.). Punctele cu probabilitate mică sunt considerate

anomalii. Avantajul principal este interpretabilitatea matematică, dar metodele parametrice eșuează când distribuția reală a datelor este necunoscută sau multimodală — situație frecventă în datele senzoriale din locuințe.

Metode bazate pe distanță și densitate. *Local Outlier Factor* (LOF), propus de Breunig et al. [5], compară densitatea locală a unui punct cu cea a vecinilor săi. LOF este eficient pentru clustere de densități diferite, dar costul computațional $\mathcal{O}(n^2)$ îl face nepotrivit pentru inferență în timp real pe dispozitive edge cu resurse limitate.

One-Class SVM. Scholkopf et al. [35] propun o variantă de *Support Vector Machine* (SVM) antrenată exclusiv pe date normale, care delimitează o frontieră de decizie în spațiul de caracteristici. Performanța depinde critic de alegerea kernelului și a hiperparametrilor, iar costul de inferență crește cu numărul de vectori suport.

Metode de izolare. Liu, Ting și Zhou [24] introduc *Isolation Forest*, un algoritm nesupervizat care izolează punctele anormale prin partiționări aleatoare succesive ale spațiului de caracteristici. Ideea centrală este că un punct anomal — rar și îndepărtat — va fi separat de restul printr-un număr mic de partiții, în timp ce un punct normal, aflat în zona densă, necesită multe partiții pentru a fi izolat. Scorul de anomalie este invers proporțional cu lungimea medie a căii de izolare. Liu et al. [25] confirmă eficiența algoritmului pe seturi de date de mari dimensiuni, cu complexitate $\mathcal{O}(n \log n)$ la antrenare și $\mathcal{O}(\log n)$ la inferență.

Metode de învățare profundă. Chalapathy și Chawla [6] trec în revistă utilizarea rețelilor neuronale profunde (autoencoder, LSTM, GAN) pentru detecția de anomalii. Aceste metode ating performanțe superioare pe seturi mari de date cu structuri complexe, dar necesită resurse computaționale semnificative (GPU, memorie RAM de ordinul gigaoctetilor) și date etichetate sau volume mari de antrenare, cerințe incompatibile cu un nod edge de tip Raspberry Pi 3B+.

Cook et al. [9] și Fahim și Sillitti [15] examinează specific tehnicile de detecție a anomaliilor în fluxuri IoT, subliniind necesitatea unor metode incrementale sau cu footprint redus. Concluzia lor converge cu alegerea Isolation Forest pentru sisteme cu constrângeri de resurse.

2.4.2 Principiul izolării — ilustrare conceptuală

Figura 2.1 ilustrează intuitiv diferența dintre comportamentul unui punct normal și cel al unui punct anomal în algoritmul Isolation Forest. Un punct anomal, aflat în afara

norului dens de observații normale, este separat prin numai câteva partiționări aleatoare. Prin contrast, un punct normal, înconjurat de alți vecini apropiați, necesită un număr mult mai mare de partiții înainte de a fi izolat. Această asimetrie stă la baza scorului de anomalie: cu cât calea de izolare este mai scurtă, cu atât probabilitatea ca observația să fie o anomalie este mai mare.

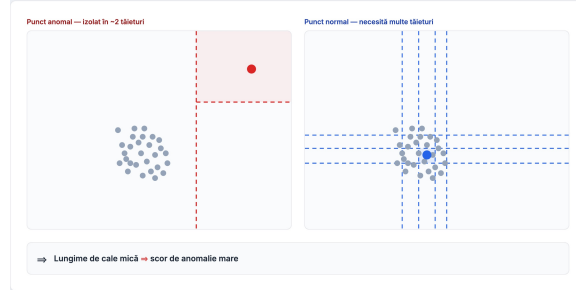


Figura 2.1: Principiul izolării: un punct anomal este separat prin puține partiționări aleatoare, în timp ce un punct normal (din zona densă) necesită multe partiții.

2.4.3 Justificarea alegerii Isolation Forest pentru sistem

Alegerea Isolation Forest ca algoritm de bază al motorului ML din această lucrare este motivată de trei proprietăți esențiale, deosebit de relevante pentru mediile edge:

1. **Nesupervizat.** Nu necesită date etichetate cu anomalii cunoscute — o cerință imposibil de îndeplinit în practică pentru o locuință nouă, unde anomaliile reale sunt rare și nedefinite a priori.
2. **Eficiență computațională.** Complexitatea $\mathcal{O}(\log n)$ la inferență [25] permite scorirea în timp real a fiecărui set de citiri senzoriale pe un Raspberry Pi 3B+ (ARMv8, 1 GB RAM), fără a afecta restul serviciilor (broker MQTT, baza de date, API).
3. **Consum redus de memorie.** Modelul serializat (`isolation_forest.pkl`) ocupă câțiva zeci de kiloocteți, spre deosebire de rețelele neuronale profunde care pot necesita sute de megaocteți. Implementarea din scikit-learn [31] este matură, testată exhaustiv și bine documentată.

Pe lângă aceste argumente tehnice, Isolation Forest s-a dovedit robust pe date multidimensionale cu distribuții arbitrare [24], ceea ce este important deoarece vectorul de caracteristici al sistemului combină valori continue (temperatură, umiditate, gaz), valori normalizate logaritmice (luminozitate) și variabile binare/ciclice (oră din zi, zi din săptămână, indicator noapte/weekend).

2.5 Poziționarea lucrării față de stadiul actual

Trecerea în revistă de mai sus permite identificarea clară a nișei pe care o ocupă sistemul propus față de soluțiile existente.

Față de platformele comerciale (Alexa, Google Home, HomeKit), sistemul propus elimină dependența de cloud și garantează că niciun datum personal nu părăsește rețeaua locală. Funcționarea completă în regim offline și costul de infrastructură zero (nu există abonamente sau servicii de hosting) constituie avantaje practice concrete pentru utilizatorul final, în spiritul principiilor local-first formalizate de Kleppmann et al. [21].

Față de platformele open-source generice (Home Assistant, openHAB), sistemul propus integrează nativ un motor de detecție a anomaliilor comportamentale antrenat pe datele specifice locuinței, nu un sistem de reguli fix sau o integrare externă opțională. Această abordare permite identificarea comportamentelor anormale subtile — de exemplu, un model de temperatură neobișnuit la o anumită oră din zi — imposibil de captat prin simple praguri hardcodate.

Față de lucrările de cercetare pe detecția de anomalii IoT [9, 15], contribuția principală a prezentei lucrări constă în implementarea completă end-to-end a unui pipeline funcțional pe hardware real de cost redus: de la colectarea datelor senzoriale printr-un nod ESP32, prin stocarea în TimescaleDB, la antrenarea zilnică automată a unui model Isolation Forest și inferența în timp real pe Raspberry Pi 3B+. Lucrările citate tratează frecvent problema detecției de anomalii în izolare, fără a aborda provocările de integrare sistem sau constrângerile edge specifice.

Noutatea esențială a lucrării este combinarea a trei caracteristici rareori întâlnite simultan în literatura de specialitate: (1) arhitectură complet *local-first*, fără nicio dependență de servicii externe; (2) detecție de anomalii comportamentale prin ML ne-supervizat (*Isolation Forest*), antrenat pe profilul real al locuinței; (3) implementare pe hardware de consum redus și cost accesibil (ESP32 + Raspberry Pi), cu o interfață mobilă React Native pentru utilizatorul final.

Capitolul următor (Capitolul 3) detaliază arhitectura sistemului și deciziile de proiectare care decurg din analiza prezentată mai sus.

3 Metodologie și arhitectura sistemului

Capitolul de față descrie principiile de proiectare care au ghidat construcția sistemului, arhitectura de ansamblu pe cinci straturi, cerințele funcționale și nefuncționale, fluxul de date bazat pe MQTT, modelul relațional al bazei de date, alegerile tehnologice justificate și, în final, metodologia contribuției originale — pipeline-ul de detectare a anomaliilor comportamentale cu algoritmul Isolation Forest.

3.1 Principii de proiectare

Arhitectura sistemului a fost concepută respectând trei principii fundamentale care influențează fiecare decizie tehnologică.

Local-first. Kleppmann et al. definesc paradigma *local-first* ca proiectarea în care toate datele utilizatorului sunt stocate și procesate pe dispozitivul sau infrastructura locală, conexiunea la cloud reprezentând o opțiune auxiliară și nu o cerință de funcționare [21]. Sistemul implementat respectă integral această paradigmă: senzoraarea, stocarea în IoT, inferența ML și controlul actuatorilor se desfășoară exclusiv în rețeaua locală (LAN), fără niciun trafic spre servicii externe. Consecința directă este că sistemul continuă să funcționeze normal chiar dacă conexiunea la Internet este absentă.

Privacy by design. Principiile *privacy by design*, consolidate în Regulamentul European GDPR [13], impun ca confidențialitatea să fie integrată în arhitectură de la etapa de proiectare, și nu adăugată ulterior. Datele de senzori — temperaturi, prezență, niveluri de gaz — sunt date cu caracter personal în sensul GDPR atunci când permit identificarea activității ocupanților. Prin stocarea exclusiv locală, sistemul elimină riscul de scurgere de date prin furnizori de cloud terți, respectând principiul minimizării transferului de date.

Edge intelligence. Shi et al. propun modelul de calcul *edge computing* ca alternativă la centralizarea în cloud, prin deplasarea puterii de calcul cât mai aproape de sursa datelor [37]. În sistemul de față, motorul ML (Isolation Forest) rulează pe Raspberry Pi 3B+ — dispozitivul gateway — și nu pe un server cloud, obținând latență de inferență de ordinul milisecundelor și independență operațională completă față de Internet.

3.2 Arhitectura de ansamblu

Sistemul este structurat în cinci straturi ierarhice, de la hardware la interfața utilizator. Figura 3.1 prezintă arhitectura de ansamblu; paragrafele de mai jos descriu fiecare strat.

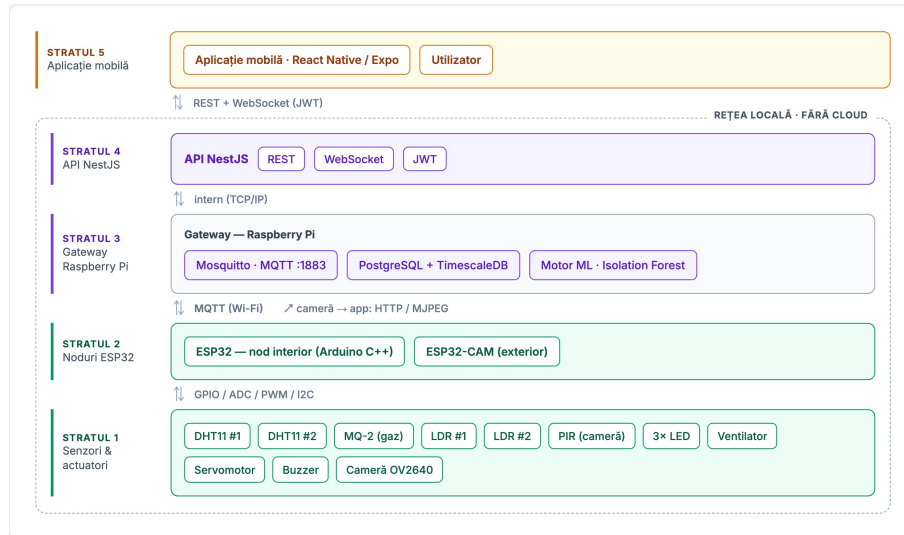


Figura 3.1: Arhitectura de ansamblu a sistemului, organizată pe cinci straturi. Toate componentele operează exclusiv în rețeaua locală, fără dependență de cloud.

Stratul 1 — Senzori și actuatori. Stratul fizic cuprinde senzorii de mediu și actuatorii conectate la nodurile ESP32. Senzorii măsoară: temperatura și umiditatea (2× DHT11), concentrația de gaze combustibile (MQ-2, ieșire analogică), nivelul de luminozitate (2× LDR) și prezența umană (un singur senzor PIR, montat pe camera exterioară). Actuatorii includ: 3 LED-uri de iluminat, un ventilator de 5V, un servomotor SG90 (pentru controlul draperiilor) și un buzzer activ pentru alerte sonore locale. Camera OV2640 (montată pe ESP32-CAM) furnizează flux video MJPEG.

Stratul 2 — Noduri ESP32. Două plăci de dezvoltare gestionează interfața cu hardware-ul fizic:

- *Nodul interior* (ESP32 WROOM-32) execută firmware Arduino C++ și comunică prin General-Purpose Input/Output (GPIO), Analog-to-Digital Converter (ADC) (10 biți, 0–1023) și Pulse-Width Modulation (PWM) cu toți senzorii și actuatorii de interior.
- *Nodul exterior* (ESP32-CAM, AI-Thinker, cu senzor de imagine OV2640) detectează mișcarea prin PIR pe GPIO13 și oferă server HTTP pe portul 80 pentru streaming video (GET /stream) și captură JPEG (GET /capture).

Ambele noduri comunică cu gateway-ul prin MQTT peste Wi-Fi; camera este descoperită de aplicația mobilă prin mDNS (smarthome-cam.local).

Stratul 3 — Gateway Raspberry Pi. Raspberry Pi 3B+ îndeplinește rolul central al sistemului. Găzduiește:

- brokerul Mosquitto ([MQTT](#) 1883, autentificat), care primește și rutează toate mesajele publicate de noduri;
- baza de date PostgreSQL cu extensia TimescaleDB, pentru stocarea eficientă a seriilor temporale de citiri;
- motorul [ML](#) Python (Isolation Forest), care antrenează și rulează inferența de detectare a anomaliilor direct pe dispozitivul edge.

Stratul 4 — [API NestJS](#). Serviciul backend, scris în TypeScript cu framework-ul NestJS, expune un [API REST](#) și un server WebSocket (Socket.IO) pe portul 3000. Comunicarea cu brokerul [MQTT](#) se realizează prin modulul `mqtt.service.ts`, care abonează NestJS la topicurile de senzori și publică comenzi spre noduri. Autentificarea este realizată cu [JWT](#) și parole *hash*-uite cu `bcrypt`.

Stratul 5 — Aplicație mobilă. Interfața utilizatorului este o aplicație React Native (Expo SDK 54, TypeScript) ce rulează pe Android și iOS. Primește date în timp real prin Socket.IO, afișează valorile senzorilor, starea alertelor și permite controlul actualelor. Descoperirea automată a gateway-ului se face prin [mDNS](#) (`smarthome.local`).

3.3 Cerințe funcționale și nefuncționale

3.3.1 Cerințe funcționale

Cerințele funcționale au fost identificate pornind de la scenariile de utilizare reale ale unui locatar. Figura 3.2 prezintă diagrama use-case a sistemului.



Figura 3.2: Diagrama use-case a sistemului smart home. Actorul principal este utilizatorul, iar motorul ML participă ca actor secundar (sistem) prin generarea automată de alerte de anomalie.

Principalele cerințe funcționale sunt:

1. **Monitorizare senzori în timp real** — sistemul colectează și afișează temperaturi (2 camere), umiditate (2 camere), nivel de gaz, luminozitate (2 senzori) și prezență, cu actualizare la fiecare 5 secunde.
2. **Control actuatori de la distanță** — utilizatorul poate acționa de pe telefon: LED-urile (pornit/oprit/toggle individual), ventilatorul, servomotorul draperiilor ($0^\circ/90^\circ/180^\circ$), buzzer-ul și comanda `all_off`.
3. **Sistem de alerte** — alertele de tip `GAS_DETECTED`, `MOTION_NIGHT`, `ALERT_HEAT`, `ALERT_COLD` și `ML_ANOMALY` sunt stocate în baza de date și notificate în timp real prin WebSocket.
4. **Vizualizare istoric** — aplicația permite consultarea istoricului grafic al citirilor pe intervale configurabile.
5. **Flux video exterior** — aplicația afișează stream MJPEG live și captură snapshot de la camera exterioară.
6. **Detectare anomalii comportamentale** — motorul `ML` local generează alerte de tip `ML_ANOMALY` pentru comportamente neobișnuite față de pattern-ul zilnic al locatarului.
7. **Configurare Wi-Fi (provisioning)** — nodul poate primi noile credențiale Wi-Fi prin comanda `MQTT wifi_update` sau, ca fallback, prin hotspot-ul `SmartHome-Setup`.

3.3.2 Cerințe nefuncționale

- **Disponibilitate locală** — sistemul funcționează integral fără conexiune la Internet; nu există dependențe de servicii cloud externe.
- **Latență redusă** — de la publicarea datelor de senzori până la afișarea în aplicație, latența end-to-end vizată este sub 2 secunde.
- **Confidențialitate** — nicio dată de senzori sau video nu părăsește rețeaua locală.
- **Extensibilitate** — arhitectura permite adăugarea de noi noduri ESP32 fără modificări în backend, prin convenția de topicuri `smarthome/ {nodeId}/sensors`.
- **Reziliență** — nodurile ESP32 implementează reconectare automată la Wi-Fi și MQTT; LWT asigură notificarea automată a deconectărilor.

3.4 Fluxul de date și protocolul MQTT

MQTT este protocolul de transport ales pentru comunicarea dintre noduri și gateway. Standardul [Organization for the Advancement of Structured Information Standards \(OASIS\)](#) MQTT 3.1.1 [30] definește un protocol publish/subscribe binar, cu overhead minim, potrivit pentru dispozitive cu resurse limitate [29]. Brokerul utilizat este Mosquitto [22], rulând cu autentificare bazată pe parolă pe portul 1883.

Figura 3.3 prezintă fluxul complet de date din sistem.

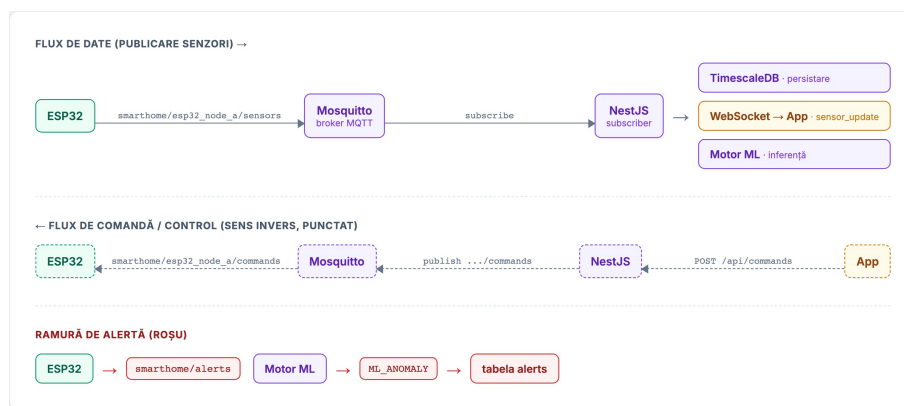


Figura 3.3: Fluxul de date în sistem. Datele de senzori circulă de la ESP32 prin Mosquitto spre NestJS, care le persistă în TimescaleDB, le transmite aplicației prin WebSocket și le evaluează prin motorul ML. Comenzile urmează traseul invers (linie punctată).

3.4.1 Topicuri MQTT și contractul de date

Tabelul 3.1 detaliază topicurile MQTT utilizate, direcția fluxului, nivelul QoS și rolul fiecăruia.

Tabelul 3.1: Topicurile MQTT ale sistemului smart home.

Topic	Direcție	QoS	Descriere
smarthome/ {nodeId}/ sensors	ESP32 → broker	0	Citiri periodice (la 5 s) în format JSON; payload conține temperaturi, umiditate, gaz, luminozitate.
smarthome/ {nodeId}/ status	ESP32 → broker	1, retained	Starea nodului: online la conectare, offline prin Last Will (LWT).
smarthome/ {nodeId}/ commands	broker → ESP32	1	Comenzi pentru actuatori: led_on , led_off , led_toggle , fan_on , fan_off , servo_move , buzzer_beep , all_off , wifi_update .
smarthome/alerts	ESP32 / ML → broker	1	Alerte de urgență: gaz detectat (de pe nod) sau anomalie ML (de pe gateway).

Payloadul publicat de nodul interior pe topicul de senzori este un obiect JSON cu câmpurile prezentate în fragmentul de mai jos (contract comun între firmware, backend și aplicația mobilă):

```

1 {
2   "nodeId":      "esp32_node_a",
3   "location":    "interior",
4   "temperature": 22.5,    // DHT11 #1, living (°C)
5   "humidity":    55,      // DHT11 #1, living (%RH)
6   "temperature2": 21.8,   // DHT11 #2, dormitor (°C)
7   "humidity2":    58,      // DHT11 #2, dormitor (%RH)
8   "gasLevel":    245,     // MQ-2 ADC 0-1023
9   "gasAlert":    false,   // true dacă gasLevel > 350
10  "light1":       410,     // LDR #1 ADC 0-1023
11  "light2":       120,     // LDR #2 ADC 0-1023
12  "lightLux":     410      // alias light1 (compatibilitate)
13 }
```

Listarea 3.1: Payload MQTT publicat de nodul interior la fiecare 5 secunde.

3.4.2 Last Will and Testament și QoS

La conectare, nodul configurează mesajul *Last Will and Testament* (LWT) pe topicul `smarthome/{nodeId}/status` cu payload `{"status": "offline"}` și flag *retained*. Dacă conexiunea TCP se întrerupe fără deconectare explicită, brokerul publică automat mesajul LWT, permițând NestJS să marcheze nodul ca offline în baza de date și să notifice aplicația mobilă prin WebSocket.

Datele de senzori se publică cu QoS 0 (fire-and-forget), acceptabil pentru telemetrie continuă cu cadență mare (pierderea unui pachet din 720 de zi este neglijabilă). Comenzile și alertele folosesc QoS 1 (cel puțin o dată), asigurând livrarea confirmată.

3.4.3 Descoperire prin mDNS și provisioning Wi-Fi

Descoperirea automată a broker-ului și a camerei utilizează [mDNS](#) (RFC 6762 [8]): ESP32 rezolvă `smarthome.local` la adresa IP curentă a Raspberry Pi, fără configurare manuală. Camera exterioară este disponibilă la `smarthome-cam.local`.

Provisioningul Wi-Fi urmează o schemă cu fallback: credențialele curente (SSID, parolă, adresa broker) sunt persistate în memoria NVS (*Non-Volatile Storage*) a ESP32. La pornire, nodul încearcă conectarea la rețeaua salvată; în caz de eșec, revine automat la hotspot-ul de configurare `SmartHome-Setup` (IP fix 192.168.4.1). Credențialele pot fi actualizate în orice moment prin comanda [MQTT](#) `wifi_update {ssid, password, mqtt_host}`, după care nodul repornește și se reconectează la noua rețea.

3.5 Modelul de date

Baza de date rulează pe PostgreSQL [38] cu extensia TimescaleDB [39], specializată în stocarea și interogarea eficientă a seriilor temporale [19]. Figura 3.4 prezintă diagrama entitate-relație.

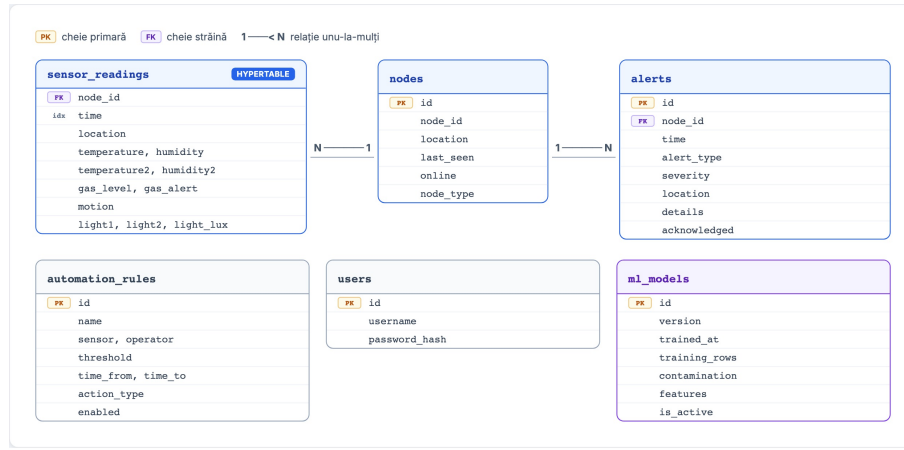


Figura 3.4: Diagrama entitate-relație a bazei de date. Tabelul `sensor_readings` este hypertable TimescaleDB partitionat automat pe coloana `time`.

Tabelul `sensor_readings` (hypertable). Este tabelul central al sistemului, convertit în hypertable TimescaleDB cu partiționare automată pe coloana `time`. Stocază fiecare citire publicată de un nod ESP32:

- `time` — `TIMESTAMPTZ`, coloana de partiționare;
- `node_id`, `location` — identificatorul și locația nodului;
- `temperature`, `humidity` — citirile DHT11 #1 (living);
- `temperature2`, `humidity2` — citirile DHT11 #2 (dormitor), `NULLABLE` (câmpuri absente în citiri mai vechi sau în caz de eroare senzor);
- `gas_level`, `gas_alert` — valoarea [ADC](#) MQ-2 și indicatorul de alertă;
- `light1`, `light2`, `light_lux` — valorile celor două [LDR](#)-uri și alias-ul de compatibilitate.

TimescaleDB comprimă automat chunk-urile mai vechi de 7 zile și permite interogări de agregare temporală cu funcții native (`time_bucket`, `first`, `last`) considerabil mai rapide decât PostgreSQL simplu pe seturi mari de date [39].

Tabelul `alerts`. Stocază alertele generate, cu câmpurile: `id`, `time`, `node_id`, `alert_type` (unul dintre `GAS_DETECTED`, `MOTION_NIGHT`, `ALERT_HEAT`, `ALERT_COLD`, `ML_ANOMALY`), `severity`, `location`, `details` (JSON cu context), `acknowledged` (boolean).

Tabelul `nodes`. Registrul nodurilor active: `id`, `node_id`, `location`, `last_seen`, `online`, `node_type`.

Tabelul `automation_rules`. Permite definirea de reguli de automatizare configurabile: senzor, operator, prag (`threshold`), interval orar (`time_from`, `time_to`), tip de acțiune (`action_type`), stare activă.

Tabelele `users` și `ml_models`. `users` păstrează credențialele autentificate (`username`, `password_hash` produs prin `bcrypt`). `ml_models` înregistrează metadatele fiecărui model antrenat: versiune (formatul `vAAAAAMZZ_HHMM`), numărul de rânduri de antrenare, valoarea `contamination`, lista feature-urilor și câmpul `is_active` care identifică modelul curent utilizat în inferență.

3.6 Alegeri tehnologice justificate

Tabelul [3.2](#) sintetizează principalele alegeri tehnologice și justificarea lor în raport cu cerințele sistemului.

Tabelul 3.2: Alegeri tehnologice — cerință, soluție adoptată și justificare.

Cerință	Tehnologie	Justificare
Microcontroler IoT cu Wi-Fi	ESP32 WROOM-32 [11]	Dual-core 240 MHz, 520 KB SRAM, Wi-Fi 802.11b/g/n integrat, suport Arduino; cost redus (~5 USD); ADC 12-biți pe mai mulți pini.
Transport mesaje pub/sub	MQTT 3.1.1 / Mosquitto [30, 22]	Protocol binar cu overhead minim (~2 B header); QoS 0–2; LWT pentru detectarea deconectărilor; potrivit pentru rețele locale cu lățime de bandă redusă.
Stocare serii temporale	PostgreSQL + TimescaleDB [39, 38]	Partiționare automată (hypertable) pe timp; compresie nativă; funcții de agregare temporală optimizate; interoperabilitate SQL deplină cu TypeORM.
Backend API	NestJS + TypeScript [28]	Arhitectură modulară cu injecție de dependențe; integrare nativă WebSocket (Socket.IO); TypeORM pentru acces la baza de date; suport JWT din cutie.
Aplicație mobilă	React Native / Expo [27, 14]	Cod TypeScript partajat între iOS și Android; Expo SDK 54 simplifică build-ul; ecosistem bogat (React Query, Zustand, Socket.IO client).
Detectare anomalii	Isolation Forest [24, 25]	Algoritm nesupervizat, fără etichete; complexitate de antrenare $O(n \log n)$; consum de memorie redus (~4 MB model.pkl); implementare matură în scikit-learn [31]; rulabil pe Raspberry Pi 3B+ fără accelerator hardware.
Autentificare	JWT bcrypt [20, 33]	Tokenuri fără stare, ușor de validat la fiecare request REST; bcrypt cu factor de cost adaptabil pentru hashuirea parolelor.

3.7 Pipeline-ul de detectare a anomaliilor — contribuția originală

Contribuția originală centrală a lucrării este un pipeline de detectare a anomaliilor comportamentale bazat pe algoritmul Isolation Forest, implementat în Python și rulând local pe Raspberry Pi 3B+. Spre deosebire de sistemele care transmit datele la cloud pentru analiză, abordarea propusă menține toate datele și calculele în rețeaua locală, respectând principiile de confidențialitate enunțate în Secțiunea 3.1.

Figura 3.5 prezintă structura completă a pipeline-ului.

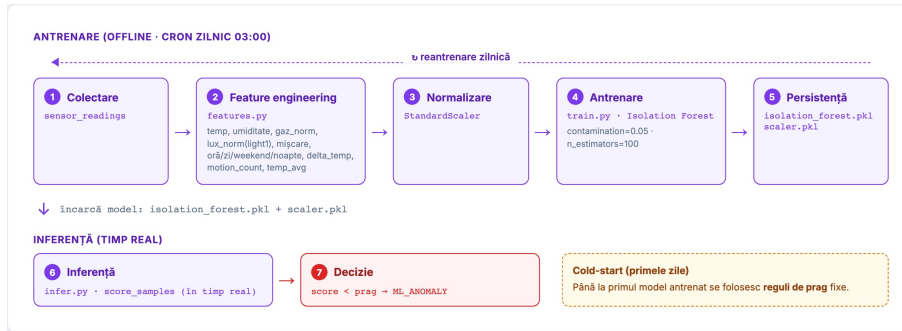


Figura 3.5: Pipeline-ul complet al motorului ML: de la colectarea datelor din TimescaleDB, prin ingineria trăsăturilor și normalizare, până la antrenarea periodică a modelului Isolation Forest și inferența în timp real. Bucla de reantrenare zilnică și mecanismul de cold-start sunt evidențiate.

3.7.1 Algoritmul Isolation Forest

Isolation Forest a fost propus de Liu, Ting și Zhou [24] și extins ulterior [25] ca algoritm de detectare a anomaliilor bazat pe proprietatea că punctele anormale sunt mai ușor de „izolat” prin partiționări aleatoare ale spațiului de trăsături față de punctele normale. Formal, un arbore de izolare (*iTree*) partiționează recursiv un subsamplu de date prin selecție aleatoare a unei trăsături și a unui prag de separare. Lungimea medie a căii de la rădăcină până la un punct $\mathbf{x}$ în toți arborii constituie scorul de anomalie.

Scorul de anomalie $s(\mathbf{x}, n)$ este definit ca:

$$s(\mathbf{x}, n) = 2^{-\frac{E[h(\mathbf{x})]}{c(n)}}, \quad (3.1)$$

unde $h(\mathbf{x})$ este lungimea căii parcurse de $\mathbf{x}$ în arbore, $E[\cdot]$ este media peste toți arborii din ansamblu, n este dimensiunea subsamplului de antrenare și $c(n) = 2H(n-1) - \frac{2(n-1)}{n}$ este lungimea medie a căii pentru n noduri (H — număr armonic). Un scor

$s \rightarrow 1$ indică o anomalie pronunțată (cale scurtă), iar $s \rightarrow 0$ indică un punct normal. Implementarea din scikit-learn [31] returnează `score_samples(x)` ca valori negative, cu semnificația: cu cât scorul este mai negativ, cu atât punctul este mai anormal.

3.7.2 Ingineria trăsăturilor (`features.py`)

Modulul `features.py` construiește 12 trăsături numerice din citirile brute ale senzorilor, în ordinea fixă din Tabelul 3.3. Ordinea este identică în antrenare și în inferență — orice modificare necesită reantrenare completă a modelului.

Tabelul 3.3: Cele 12 trăsături ale modelului Isolation Forest, în ordinea fixă din `FEATURE_NAMES`.

#	Trăsătură	Descriere și sursă
0	<code>temperature</code>	Temperatura DHT11 #1 (living), °C.
1	<code>humidity</code>	Umiditatea relativă DHT11 #1 (%RH).
2	<code>gas_normalized</code>	Valoarea ADC MQ-2 împărțită la 4095 (normalizare în $[0, 1]$).
3	<code>lux_normalized</code>	$\log_{10}(\text{light1})/7,0$ — luminozitate LDR #1 normalizată logaritmic pe scala ADC 0–1023 ($\log_{10}(1023) \approx 6,93$).
4	<code>motion</code>	Prezența umană detectorului PIR (0/1).
5	<code>hour_of_day</code>	Ora din zi (0–23), captează ritmul circadian.
6	<code>day_of_week</code>	Ziua săptămânii (0 = luni, 6 = duminică).
7	<code>is_weekend</code>	1 dacă ziua este sâmbătă sau duminică.
8	<code>is_night</code>	1 dacă ora ≥ 23 sau ora < 6 .
9	<code>temp_delta_1h</code>	Diferența de temperatură față de citirile cu 2 eșantioane în urmă (aproximativ 10 s).
10	<code>motion_count_10</code>	Suma evenimentelor de mișcare în ultimele 10 citiri (fereastră glisantă).
11	<code>temp_avg</code>	Media celor doi senzori DHT11; dacă DHT11 #2 lipsește, se folosește valoarea DHT11 #1.

3.7.3 Normalizare

Înainte de antrenare, trăsăturile sunt normalizate cu `StandardScaler` din `scikit-learn`, care transformă fiecare trăsătură la medie 0 și deviație standard 1:

$$z_i = \frac{x_i - \mu_i}{\sigma_i}. \quad (3.2)$$

Parametrii μ_i și σ_i sunt calculați pe datele de antrenare și salvați în `scaler.pkl` alături de model. La inferență, aceeași transformare este aplicată fiecărei noi citiri înainte de evaluarea scorului.

3.7.4 Antrenarea modelului (`train.py`)

Modulul `train.py` implementează ciclul de antrenare:

1. Citește din tabela `sensor_readings` toate înregistrările din ultimele 30 de zile cu `gas_alert = FALSE` (citeșterile din perioadele de alertă sunt excluse pentru a nu contamina distribuția de normalitate).
2. Verifică pragul minim de 500 de rânduri; dacă datele sunt insuficiente, antrenarea este întreruptă cu mesaj informativ.
3. Apelează `build_features(df)` și normalizează cu `StandardScaler`.
4. Antrenează un `IsolationForest` cu parametrii: `n_estimators=100`, `contamination=0.05`, `random_state=42`, `n_jobs=-1` (toate nucleele disponibile).
5. Salvează `isolation_forest.pkl` și `scaler.pkl` în directorul `models/`.
6. Înregistrează metadatele modelului nou în tabela `ml_models` și marchează toate versiunile anterioare ca inactive.

Parametrul `contamination=0.05` afirmă că modelul estimează că 5% din datele de antrenare reprezintă comportamente atipice. Această valoare este conservatoare — subestimarea duce la mai puține fals-negative în faza de deploy. Reantrenarea este planificată zilnic la ora 03:00 prin `scheduler.py`, rulat ca serviciu systemd `smarthome-ml.service` pe Raspberry Pi.

3.7.5 Inferența în timp real (`infer.py`)

Clasa `AnomalyDetector` din `infer.py` este instanțiată la pornirea serviciului NestJS și menținută în memorie. La fiecare citire nouă de senzori:

1. Se construiește un DataFrame cu o singură linie din payload-ul JSON primit.
2. Se aplică `build_features` și `scaler.transform`.
3. Se apelează `model.score_samples(X_scaled)`, care returnează un scalar real; notăm scorul ca $s \in \mathbb{R}$.
4. Dacă $s < \tau$ (unde $\tau = -0,1$ este pragul `ANOMALY_THRESHOLD`), citirea este clasificată drept anomalie și se creează o alertă de tip `ML_ANOMALY` în baza de date, notificată imediat aplicației prin WebSocket.

Metoda returnează un obiect cu câmpurile `is_anomaly` (boolean), `score` (valoare numerică), `threshold` și `model_active`. Dacă fișierele `.pkl` nu există, câmpul `model_active` este `false` și nicio alertă ML nu este generată.

3.7.6 Mecanismul de cold-start

În primele aproximativ 14 zile de funcționare a sistemului, nu există suficiente date pentru a antrena modelul (minimul de 500 de rânduri echivalând cu circa 42 de minute de date, dar o fereastră de 30 de zile este necesară pentru a surprinde variabilitatea comportamentală zilnică și săptămânală). În această perioadă de cold-start, motorul ML returnează `model_active: false` și sistemul se bazează pe reguli de prag hardcodate pentru generarea alertelor:

- `gas_level > 300` → alertă `GAS_DETECTED`;
- `temperature > 35°C` → alertă `ALERT_HEAT`;
- `temperature < 5°C` → alertă `ALERT_COLD`;
- mișcare detectată în intervalul 23:00–06:00 → alertă `MOTION_NIGHT`.

Această strategie asigură că sistemul oferă protecție de bază chiar înainte ca modelul ML să fie disponibil.

3.8 Considerații de securitate

Modelul de securitate al sistemului este construit pe trei niveluri complementare.

Autentificare și autorizare. Accesul la API-ul REST și la conexiunile WebSocket este protejat prin JWT [20]. La autentificarea reușită, serverul emite un token semnat

cu o cheie secretă; toate request-urile ulterioare trebuie să includă tokenul în header-ul `Authorization: Bearer <token>`. Parolele utilizatorilor sunt stocate exclusiv ca hash-uri bcrypt [33] cu factor de cost adaptabil (implicit 10 runde), eliminând riscul expunerii parolilor în clar chiar în cazul unui acces neautorizat la baza de date.

Autentificare broker MQTT. Brokerul Mosquitto este configurat cu autentificare prin user/parolă (utilizatorul `smarthome`, credențiale stocate în fișierul Mosquitto `passwd`), permițând conectarea doar nodurilor și serviciilor cunoscute. Comunicarea se desfășoară în rețeaua locală și nu este expusă direct la Internet.

Izolare în rețeaua locală. Nicio componentă a sistemului nu expune porturi pe Internet. Gateway-ul Raspberry Pi, nodurile ESP32 și aplicația mobilă comunică exclusiv în LAN. Această izolare reprezintă o măsură de securitate perimetrală complementară celor aplicative, conformă cu principiul *privacy by design* din GDPR [13]. Datele cu caracter personal (activitate comportamentală, prezență umană) nu părăsesc niciodată rețeaua locală, eliminând riscurile asociate transferului transfrontalier de date sau breșelor de securitate la nivel de furnizor cloud.

Minimizarea suprafeței de atac. Firmware-ul ESP32 nu expune nicio interfață HTTP sau Telnet în mod normal de funcționare; singurul vector de comunicare externă este conexiunea MQTT autentificată. ESP32-CAM expune server HTTP pe portul 80 exclusiv în rețeaua locală, accesul nefiind publicat prin NAT sau port-forwarding.

4 Implementarea prototipului

Capitolul de față descrie concret modul în care componentele prezentate în Capitolul 3 au fost transpuse în cod funcțional și asamblate într-un prototip operațional. Sunt tratate, în ordine, firmware-ul nodului interior bazat pe [Espressif Systems 32-bit SoC \(ESP32\)](#), firmware-ul camerei exterioare [ESP32-CAM](#), configurarea gateway-ului Raspberry Pi, structura [API](#)-ului NestJS, motorul de detecție a anomaliilor cu [IF](#) și, în final, aplicația mobilă.

4.1 Firmware-ul nodului interior (ESP32 WROOM-32)

4.1.1 Pinout și configurare hardware

Nodul interior este construit în jurul modulului ESP32 WROOM-32. Toți pinii [ADC](#) folosiți pentru senzori (MQ-2, [LDR #1](#) și [LDR #2](#)) aparțin grupului ADC1 (GPIO34, GPIO35, GPIO36), ceea ce permite citiri analogice corecte chiar și cu [Wi-Fi](#) activ — spre deosebire de pinii ADC2 care devin inutilizabili în prezența radio-ului [11]. Modulul BH1750 (citire de lux prin I²C), luat în considerare inițial, a fost eliminat în favoarea celor două [LDR](#) analogice, simplificând schema de cablare.

Tabelul 4.1 centralizează cei unsprezece pini activi ai nodului, împărțiți în senzori și actuatori.

Tabelul 4.1: Pinout complet al nodului interior (ESP32 WROOM-32).

Componentă	Pin GPIO	Observații
DHT11 #1 (living)	GPIO4	Digital, pull-up extern $10\text{ k}\Omega$ la $3,3\text{ V}$
DHT11 #2 (dormitor)	GPIO23	Digital, pull-up extern $10\text{ k}\Omega$ la $3,3\text{ V}$
MQ-2 (gaz, A0)	GPIO34	ADC1, intrare; divizor de tensiune $0\text{--}3,3\text{ V}$
LDR #1 (lumină)	GPIO35	ADC1, intrare; divizor cu $10\text{ k}\Omega$
LDR #2 (lumină)	GPIO36	ADC1 (VP), intrare; divizor cu $10\text{ k}\Omega$
Buzzer activ	GPIO5	Ieșire digitală, oscilator intern
Servo SG90 (draperii)	GPIO18	PWM 50 Hz ; alimentat din 5 V extern
LED #0 (living)	GPIO25	Ieșire digitală + rezistor serie $220\ \Omega$
LED #1 (dormitor)	GPIO26	Ieșire digitală + rezistor serie $220\ \Omega$
LED #2 (hol)	GPIO27	Ieșire digitală + rezistor serie $220\ \Omega$
Ventilator 5 V	GPIO33	Comandat prin tranzistor NPN (NU direct din GPIO)

Figura 4.1 ilustrează schema de conexiuni a nodului, cu marcarea alimentărilor separate: $3,3\text{ V}$ pentru senzori (DHT11, LDR) și 5 V extern pentru servo și ventilator.



Figura 4.1: Schema de conexiuni a nodului interior ESP32 WROOM-32 (pinout conform Tabelul 4.1).

4.1.2 Citirea senzorilor

La inițializare, rezoluția **ADC** este setată explicit la 10 biți (`analogReadResolution(10)`), oferind valori în intervalul 0–1023. Ambii senzori **DHT11** sunt citați cu biblioteca *Adafruit DHT*, iar erorile temporare (valori NaN) sunt tratate tolerant: dacă cel puțin un senzor raportează date valide, publicarea continuă cu câmpurile disponibile; numai dacă ambii eșuează simultan este omisă întreaga publicare [3].

Senzorul de gaz MQ-2 returnează o valoare analogică brută (0–1023). Când valoarea depășește pragul `GAS_THRESHOLD = 350`, firmware-ul detectează *frontul crescător* (tranziția `false` → `true`), publică o alertă pe topicul `smarthome/alerts` și activează buzzerul local cu trei bipuri — prevenind astfel spamul la broker în caz de depășire prelungită [18]. Listarea 4.1 prezintă nucleul funcției de colectare și publicare.

```

1 // Citire senzori ADC - 10 biti (0-1023)
2 int gas    = analogRead(MQ2_PIN);    // GPIO34
3 int light1 = analogRead(LDR1_PIN);   // GPIO35
4 int light2 = analogRead(LDR2_PIN);   // GPIO36
5
6 // Alerta gaz - doar pe frontul crescator
7 bool gasAlert = (gas > GAS_THRESHOLD);
8 if (gasAlert && !gasAlertActive) {
9     publishGasAlert(gas);
10    beep(3);           // semnal sonor local imediat
11 }
12 gasAlertActive = gasAlert;
13
14 // Payload JSON (contract cu backend si mobile)
15 StaticJsonDocument<384> doc;
16 doc["nodeId"]    = NODE_ID;           // "esp32_node_a"
17 doc["location"]  = LOCATION;          // "interior"
18 if (dht1ok) { doc["temperature"] = temp1; doc["humidity"] = humid1; }
19 if (dht2ok) { doc["temperature2"] = temp2; doc["humidity2"] = humid2; }
20 doc["gasLevel"]  = gas;
21 doc["gasAlert"]  = gasAlert;
22 doc["light1"]    = light1;
23 doc["light2"]    = light2;
24 doc["lightLux"]  = light1;           // alias backward-compatible

```

Listarea 4.1: Fragment din `publishSensors()` — citire DHT11, MQ-2, LDR și construire payload JSON (`esp32_node.ino`).

4.1.3 Actuatori

Ventilatorul (5 V) nu poate fi alimentat direct din **GPIO** — curentul maxim al unui pin ESP32 este de 12 mA — astfel că este comandat printr-un tranzistor NPN pe GPIO33: nivel HIGH pe pin conduce tranzistorul și închide circuitul motorului. Servo-ul SG90, folosit pentru controlul draperiilor, primește semnal **PWM** la 50 Hz prin biblioteca *ESP32Servo*; funcția `servoDraperii.write(angle)` acceptă unghiuri între 0° și 180° (corespunzând draperiilor complet deschise, respectiv complet închise).

Ventilatorul poate fi acționat în două moduri, comutabile din aplicație: *manual* (comenzile `fan_on/fan_off`) și *automat* (comanda `fan_auto`). În modul automat, firmware-ul pornește ventilatorul când temperatura medie a celor doi senzori `DHT11` depășește un **prag configurabil de la distanță** (comanda `fan_threshold`, implicit 28°C) și îl oprește sub (prag $- 2^{\circ}\text{C}$), histerezisul reducând oscilațiile rapide în jurul pragului. Atât modul, cât și pragul sunt salvate în memoria NVS și raportate înapoi în mesajul de senzori (`fanOn`, `fanAuto`, `fanThreshold`), astfel încât aplicația reflectă starea reală a dispozitivului.

4.1.4 Comunicație MQTT, provisioning Wi-Fi și mDNS

La pornire, nodul citește credențialele Wi-Fi și adresa brokerului `MQTT` din memoria NVS (*Non-Volatile Storage*) folosind biblioteca *Preferences*. Dacă rețeaua salvată nu răspunde, dispozitivul revine automat la hotspot-ul de configurare (`SmartHome-Setup`), de unde o comandă `wifi_update` — primită ulterior tot prin `MQTT` — poate instala noi credențiale și reporni nodul.

Adresa brokerului poate fi specificată ca nume mDNS (`smarthome.local`); în acest caz, funcția `resolveMqttHost()` interogază serverul mDNS local (până la cinci tentative) și returnează IP-ul corespunzător, eliminând necesitatea configurării unui IP static pe gateway [8].

Conexiunea `MQTT` este stabilită cu autentificare (utilizator și parolă) și cu un mesaj *Last Will and Testament* reținut pe topicul `smarthome/ esp32_node_a/status`: dacă brokerul detectează o deconectare bruscă, publică automat `{"node_id": "esp32_node_a", "status": "offline"}`, informând aplicația mobilă în timp real [30]. La conectare cu succes, nodul se abonează la `smarthome/ esp32_node_a/commands` și publică imediat statusul `online`. Publicarea datelor de senzori are loc la fiecare 5s pe topicul `smarthome/ esp32_node_a/sensors`.

4.2 Firmware-ul ESP32-CAM (nod exterior)

Nodul exterior este realizat pe un modul AI-Thinker ESP32-CAM echipat cu senzorul de imagine OV2640 [12]. Singurul senzor de mișcare din sistem — un `PIR HC-SR501` — este conectat la GPIO13 [40]. La detecția mișcării, un `LED` exterior conectat la GPIO12 se aprinde timp de 30s (gestionat prin variabila `lastMotion` și temporizare cu `millis()`).

4.2.1 Serverul HTTP și stream-ul MJPEG

Camera este inițializată prin `esp_camera_init()`, cu rezoluție SVGA (800×600) dacă este prezentă memoria PSRAM, sau VGA în lipsa acesteia. Firmware-ul pornește un server HTTP pe portul 80 cu două rute:

- `GET /capture` — returnează un snapshot JPEG al cadrului curent (`Content-Type: image/jpeg`);
- `GET /stream` — transmite un flux *Motion JPEG* (`multipart/x-mixed-replace`), cu o pauză de 50 ms între cadre ($\approx 15\text{--}20$ fps), lăsând procesorului timp și pentru bucla [MQTT](#).

Camera devine accesibilă în rețeaua locală ca `http://smarthome-cam.local` prin înregistrarea unui serviciu mDNS (`MDNS.begin("smarthome-cam")`), similar nodului interior. Tot periodic (la 5 s), nodul publică un mesaj JSON pe topicul `smarthome/esp32_cam_node/sensors` conținând câmpul `motion` (0/1), iar conexiunea [MQTT](#) folosește același mecanism *Last Will* ca nodul interior.

4.3 Gateway Raspberry Pi

Gateway-ul rulează pe un Raspberry Pi 3B+ și găzduiește toate serviciile de infrastructură ale sistemului.

Mosquitto este brokerul [MQTT](#) (portul 1883), configurat cu autentificare prin fișier de parole. Acesta primește publicările nodurilor ESP32, le distribuie abonaților ([API](#)-ului NestJS și motorului ML) și gestionează mesajele *retained* pentru statusul nodurilor.

PostgreSQL + TimescaleDB stochează toate citirile de senzori în hypertabela `sensor_readings`, optimizată pentru interogări time-series prin indexuri pe coloana `time` [39, 38]. Celelalte tabele ale schemei (`alerts`, `nodes`, `automation_rules`, `users`, `ml_models`) sunt tabele PostgreSQL standard.

Fiecare componentă software este înregistrată ca serviciu **systemd**, asigurând pornirea automată la boot și repornirea în caz de eroare: `smarthome-api.service` (NestJS), `smarthome-ml.service` (antrenare zilnică la 03:00) și `mosquitto.service`.

4.4 API NestJS

[API](#)-ul backend este implementat cu framework-ul NestJS (Node.js, TypeScript) și rulează pe portul 3000 al gateway-ului [28]. TypeORM asigură maparea obiect-relațională față de PostgreSQL.

4.4.1 Structura modulelor

AppModule importă nouă module funcționale: AuthModule, UsersModule, SensorsModule, AlertsModule, RulesModule, CommandsModule, MqttModule, SimulatorModule și SetupModule. Comunicarea inter-modul se realizează prin EventEmitter2 (biblioteca *@nestjs/event-emitter*), evitând dependențe circulare.

4.4.2 Subscriber MQTT

MqttService implementează OnModuleInit și se conectează la Mosquitto la pornire (dezactivat automat dacă SIMULATE=true). Se abonează la trei familii de topicuri cu wildcard: smarthome/+/sensors (date senzori), smarthome/alerts (alerte hardware) și smarthome/+/status (prezența nodurilor). Listarea 4.2 prezintă logica de procesare a mesajelor.

```

1 private async handleMessage(topic: string, payload: string) {
2   const data = JSON.parse(payload);
3
4   if (topic.endsWith('/sensors')) {
5     await this.sensorsService.saveReading({
6       nodeId:      data.nodeId ?? data.node_id,
7       temperature: data.temperature,
8       humidity:    data.humidity,
9       temperature2: data.temperature2 ?? null,
10      humidity2:    data.humidity2  ?? null,
11      gasLevel:     data.gasLevel,
12      gasAlert:     data.gasAlert ?? false,
13      light1:       data.light1    ?? null,
14      light2:       data.light2    ?? null,
15      lightLux:     data.lightLux ?? data.light1,
16      motion:       data.motion   ?? false,
17    });
18   } else if (topic === 'smarthome/alerts') {
19     await this.alertsService.create({
20       nodeId:      data.node_id,
21       alertType:   data.alert_type, // ex. "GAS_DETECTED"
22       severity:    data.alert_type === 'GAS_DETECTED' ? 'critical' : 'medium',
23       location:    data.location,
24       details:     data.details,
25     });
26   } else if (topic.endsWith('/status')) {
27     await this.sensorsService.updateNodeStatus(data);
28   }
29 }

```

Listarea 4.2: Handler de mesaje MQTT în MqttService (mqtt.service.ts) — ramificarea pe tip de topic.

Publicarea comenzilor spre noduri se face prin evenimentul `command.sent` emis de `CommandsService` și capturat de `MqttService`, care îl publică pe topicul `smarthome/{nodeId}/commands`.

4.4.3 WebSocket (Socket.IO)

`SensorsGateway` declară un namespace `Socket.IO /live` și reacționează la trei evenimente interne: `sensor.reading` → emite `sensor_update` (datele noi de senzori), `alert.new` → emite `alert`, `node.status` → emite `node_status`. La conectarea unui client, tokenul `JWT` este verificat opțional; dacă tokenul e prezent dar invalid, clientul este deconectat imediat.

4.4.4 Endpointuri REST și autentificare JWT

Toate rutele protejate sunt acoperite de garda `JwtAuthGuard`. Tabelul următor rezumă principalele endpointuri [20]:

Tabelul 4.2: Principalele endpointuri REST ale API-ului NestJS.

Metodă	Cale	Descriere
POST	<code>/api/auth/login</code>	Autentificare; returnează token JWT
GET	<code>/api/sensors/latest</code>	Ultima citire per nod
GET	<code>/api/sensors/nodes</code>	Lista nodurilor cu status online
GET	<code>/api/sensors/history</code>	Istoric cu filtre <code>nodeId</code> , <code>from</code> , <code>to</code> , <code>limit</code>
GET	<code>/api/alerts</code>	Toate alertele
POST	<code>/api/alerts/{id}/acknowledge</code>	Marchează alertă ca recunoscută
POST	<code>/api/commands/{nodeId}</code>	Trimite comandă actuator
GET	<code>/api/rules</code>	Reguli de automatizare
GET	<code>/api/setup/status</code>	Status provisionare Wi-Fi

Autentificarea utilizează JWT cu semnătură HS256. Serviciul `AuthService` verifică parola prin `bcrypt.compare()` față de hash-ul stocat în baza de date [33], iar `JwtService.sign()` emite tokenul cu payload `{sub, username}`.

4.4.5 Modulul simulator

Pentru dezvoltare fără hardware fizic, variabila de mediu `SIMULATE=true` activează `SimulatorService`. Acesta generează la fiecare 5 s date realiste pentru nodul interior (`esp32_node_a`): temperaturi de bază de 21 °C în living și 19 °C în dormitor, umiditate, gaz și

mișcare aleatorie, inclusiv câmpurile `temperature2`, `humidity2`, `light1`, `light2`, asigurând paritatea cu schema reală a payload-ului. Când `SIMULATE=false`, `MqttService` preia controlul, iar `SimulatorService` nu pornește timer-ul.

4.5 Motorul de detecție a anomaliilor cu Isolation Forest

Detecția anomaliilor comportamentale constituie contribuția originală a acestei lucrări. Motorul **ML** rulează în întregime local pe Raspberry Pi, fără dependență de cloud, în acord cu principiul *local-first* descris în Capitolul 3 [24, 31].

4.5.1 Ingineria trăsăturilor (`features.py`)

Funcția `build_features(df)` transformă un `DataFrame` Pandas cu citiri brute în matricea de trăsături necesară modelului [26]. Cele 12 trăsături, în ordinea fixă din `FEATURE_NAMES`, sunt descrise în Tabelul 4.3.

Tabelul 4.3: Cele 12 trăsături utilizate de motorul Isolation Forest.

Idx	Trăsătură	Sursă / transformare
0	<code>temperature</code>	DHT11 #1, °C
1	<code>humidity</code>	DHT11 #1, %RH
2	<code>gas_normalized</code>	MQ-2 ADC /4095
3	<code>lux_normalized</code>	$\log_{10}(\text{light1})/7,0$ (LDR #1, scara ADC)
4	<code>motion</code>	PIR 0/1 (de pe ESP32-CAM, GPIO13)
5	<code>hour_of_day</code>	Ora din zi, 0–23
6	<code>day_of_week</code>	0 (luni) – 6 (duminică)
7	<code>is_weekend</code>	1 dacă zi = sâmbătă sau duminică
8	<code>is_night</code>	1 dacă oră ≥ 23 sau oră < 6
9	<code>temp_delta_1h</code>	Diferența de temperatură față de 2 citiri anterioare
10	<code>motion_count_10</code>	Suma evenimentelor PIR în fereastra de 10 citiri
11	<code>temp_avg</code>	Media DHT11 #1 și #2 (fallback = DHT11 #1)

Trăsăturile temporale (ora, ziua, weekend, noapte) permit modelului să distingă comportamente normale specifice perioadei (ex. mișcare nocturnă sau temperaturi ridi-

cate în weekend) de adevăratele anomalii. Normalizarea `StandardScaler` (medie zero, deviație standard unu) este aplicată în `train.py` și reutilizată în `infer.py`.

Listarea 4.3 prezintă calculul trăsăturilor derivate.

```

1 df['gas_normalized'] = df['gas_level'].fillna(0) / 4095.0
2
3 # Luminozitate: LDR ADC 0-1023; log1p/7 → interval ~[0, 1]
4 if 'light1' in df.columns:
5     raw_light = df['light1'].fillna(
6         df.get('light_lux', pd.Series(0, index=df.index)).fillna(0))
7 else:
8     raw_light = df['light_lux'].fillna(0)
9 df['lux_normalized'] = np.log1p(raw_light) / 7.0
10
11 # Media DHT11 #1 si #2 (daca #2 lipseste, fallback la #1)
12 if 'temperature2' in df.columns:
13     temp2 = df['temperature2'].fillna(df['temperature'])
14 else:
15     temp2 = df['temperature']
16 df['temp_avg'] = (df['temperature'] + temp2) / 2.0

```

Listarea 4.3: Calcul trăsături derivate în `features.py` — normalizare gaz, lumină și media DHT11.

4.5.2 Antrenarea modelului (`train.py`)

Scriptul `train.py` extrage din PostgreSQL ultimele 30 de zile de citiri fără alertă de gaz (citirile de gaz anormal ar perturba distribuția normală), construiește matricea de trăsături, aplică `StandardScaler` și antrenează un `IsolationForest` cu parametrii: `contamination=0.05` (5% din date asumate anormale), `n_estimators=100`, `random_state=42` [24]. Este impus un minim de 500 de rânduri; sub acest prag, scriptul se oprește cu mesaj de informare, fără a suprascrie modelul existent.

Modelul antrenat și scalerul sunt persistate cu `joblib` (`isolation_forest.pkl`, `scaler.pkl`), iar metadatele versiunii (număr de rânduri, contaminare, lista de trăsături) sunt înregistrate în tabela `ml_models` din PostgreSQL. Reantrenarea este declanșată zilnic la 03:00 de serviciul `systemd smarthome-ml.service` prin `scheduler.py`.

4.5.3 Inferența în timp real (`infer.py`)

Clasa `AnomalyDetector` expune metoda `score(reading: dict)` care primește o citire nouă (dicționar Python), construiește un `DataFrame` de un singur rând, aplică `build_features()` și `scaler.transform()`, apoi calculează `model.score_samples(X_scaled)[0]`. Un scor sub pragul `ANOMALY_THRESHOLD = -0.1` declanșează crearea unei alerte de tip `ML_ANOMALY` în baza de date, urmată de notificarea aplicației mobile prin `WebSocket` [7].

Strategia cold-start: în primele zile de funcționare, când baza de date nu are suficiente date pentru antrenare (< 500 înregistrări), motorul revine la reguli cu prag hardcodat (gaz > 300 , temperatură $> 35^{\circ}\text{C}$ sau $< 5^{\circ}\text{C}$, mișcare în intervalul 23:00–06:00), asigurând un nivel minim de protecție până când modelul statistic poate fi inițializat. Funcția `is_ready()` permite codului apelant să distingă cele două regimuri.

4.6 Fluxuri principale — diagrame de secvență

Această secțiune descrie cele trei fluxuri principale ale sistemului prin diagrame de secvență UML.

4.6.1 Fluxul de monitorizare

Figura 4.2 ilustrează ciclul de colectare și afișare a datelor de senzori. La fiecare 5 s, ESP32 citește toți senzorii și publică un mesaj JSON pe topicul `smarthome/ esp32_ node_a/sensors`. MqttService (NestJS) recepționează mesajul, apelează `SensorsService.saveReading()` care persistă datele în TimescaleDB, emit evenimentul intern `sensor.reading`, iar SensorsGateway transmite `sensor_update` prin Socket.IO tuturor clienților conectați la namespace-ul `/live`. Aplicația mobilă actualizează dashboard-ul în timp real prin hook-ul React Query care ascultă acest eveniment.

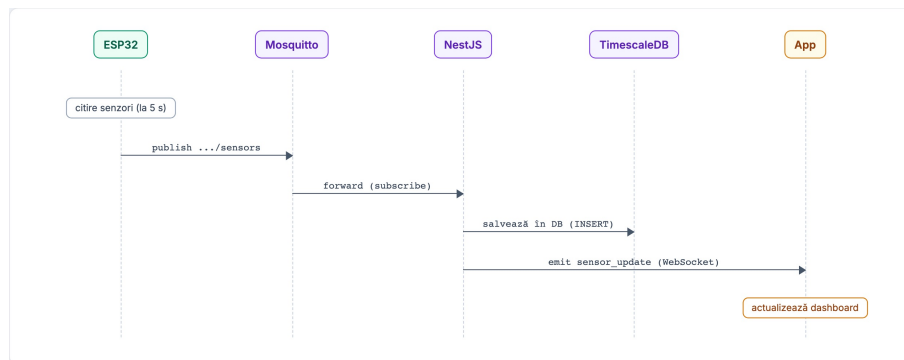


Figura 4.2: Diagrama de secvență pentru fluxul de monitorizare: ESP32 → Mosquitto → NestJS → TimescaleDB → App.

4.6.2 Fluxul de comandă a actuatorilor

Figura 4.3 descrie traseul unei comenzi de la utilizator la actuatorul fizic. Utilizatorul apasă butonul „LED 1 ON” în ecranul Control al aplicației, care trimite `POST /api/commands/esp32_node_a` cu corpul `{"action": "led_on", "led": 0}`. `CommandsController` (NestJS) procesează cererea, emite evenimentul `command.sent`, iar `MqttService` publică

comanda pe topicul `smarthome/ esp32_node_a/commands` (QoS 1). ESP32 recepționează mesajul în callback-ul `onMessage()`, apelează `setLed(0, true)` și aplică nivel HIGH pe GPIO25. Întreg fluxul se desfășoară în rețeaua locală, fără ieșire pe internet.

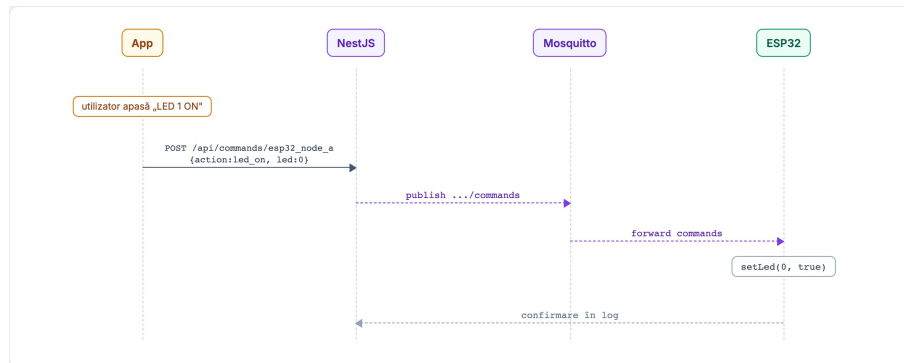


Figura 4.3: Diagrama de secvență pentru fluxul de comandă a actuatorilor: App → NestJS → Mosquitto → ESP32 → actuator fizic.

4.6.3 Fluxul de detecție a anomaliilor

Figura 4.4 prezintă traseul complet al unei anomalii detectate de motorul IF. O citire nouă sosită pe topicul `smarthome/+ /sensors` este salvată în baza de date, după care `SensorsService` apelează modulul ML (`ml.service.ts`) care invocă sincron `Anomaly-Detector.score()`. Dacă scorul returnat este sub pragul $-0,1$, `AlertsService` creează o înregistrare de tip `ML_ANOMALY` în tabela `alerts`, emit evenimentul `alert.new`, iar `SensorsGateway` îl transmite aplicației prin Socket.IO. Utilizatorul primește o notificare vizuală pe ecranul Alerte.

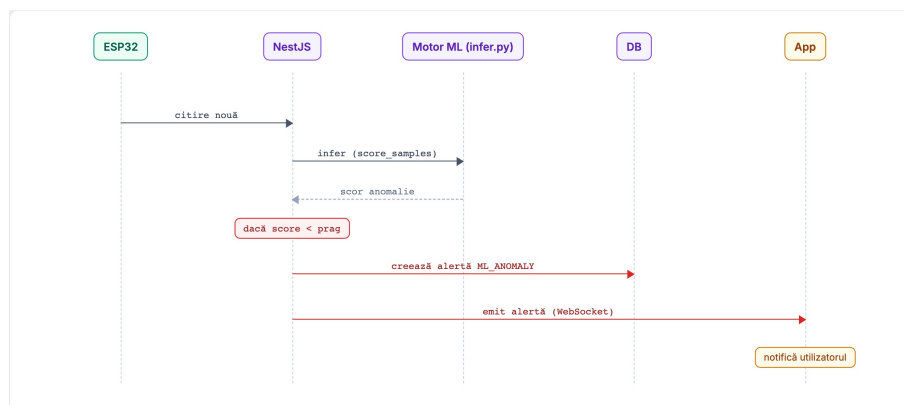


Figura 4.4: Diagrama de secvență pentru detecția anomaliei comportamentale cu IF: NestJS apelează `infer.py` și generează alertă `ML_ANOMALY`.

4.7 Aplicația mobilă

Interfața utilizatorului este implementată ca aplicație React Native 0.81 compilată cu Expo SDK 54, targetând iOS și Android [27, 14]. Starea globală este gestionată cu **Zustand**, apelurile **REST** cu **React Query**, iar datele în timp real prin **Socket.IO** conectat la namespace-ul `/live` al **API**-ului. Autentificarea utilizează token **JWT** stocat local.

4.7.1 Discovery-ul gateway-ului

La prima pornire, aplicația încearcă să contacteze gateway-ul în ordinea: variabila de mediu configurată la build-time, `http://smarthome.local:3000` (mDNS) sau adresa fixă a hotspot-ului de configurare (`192.168.4.1`). Această strategie elimină nevoia configurării manuale a IP-ului în scenariile tipice de rețea locală, susținând obiectivul de funcționare „plug and play”. Implementarea, prezentată în Listarea 4.4, sondează *în paralel* toate adresele candidate și o reține pe prima care răspunde la endpointul public `/api/setup/status`; la o eroare de rețea adresa este invalidată, astfel încât aplicația redescoperă gateway-ul atunci când telefonul trece, de exemplu, de pe hotspot pe rețeaua Wi-Fi a casei.

```

1 const CANDIDATES = [
2   API_HOST,                // variabila de mediu / fallback
3   'http://smarthome.local:3000', // mDNS - orice rețea
4   HOTSPOT_HOST,           // hotspot RPi - 192.168.4.1
5 ];
6
7 // sondeaza in paralel; prima adresa care raspunde "castiga"
8 export async function resolveHost(force = false): Promise<string> {
9   if (resolvedHost && !force) return resolvedHost;
10  return Promise.any(CANDIDATES.map(probe))
11    .then((host) => { resolvedHost = host; return host; })
12    .catch(() => API_HOST);
13 }
14
15 async function probe(host: string): Promise<string> {
16   // /api/setup/status e public si raspunde doar de pe RPi
17   await axios.get(`${host}/api/setup/status`, { timeout: 4000 });
18   return host;
19 }

```

Listarea 4.4: Descoperirea automată a gateway-ului în rețea (`services/discovery.ts`).

4.7.2 Recepția datelor în timp real (Socket.IO)

Actualizarea live a interfeței nu se bazează pe interogare repetată (*polling*), ci pe o conexiune **Socket.IO** către namespace-ul `/live` al **API**-ului, autentificată cu același token

JWT. La fiecare eveniment primit de la server, handler-ul actualizează direct store-ul Zustand, iar componentele abonate se redesenează automat (Listarea 4.5). Reconectarea este automată, cu un număr nelimitat de încercări — esențial pentru un dispozitiv mobil care își schimbă frecvent starea rețelei.

```

1 socket = io(`${host}/live`, {
2   auth: { token },
3   transports: ['websocket'],
4   reconnectionDelay: 3000,
5   reconnectionAttempts: Infinity,
6 });
7
8 // citire noua de senzor -> actualizeaza store-ul -> UI live
9 socket.on('sensor_update', (reading: SensorReading) => {
10   useAppStore.getState().setLatestReading(reading);
11 });
12 socket.on('alert', (alert: AlertItem) => {
13   useAppStore.getState().addAlert(alert);           // badge + ecran Alerte
14 });
15 socket.on('node_status', (s: { nodeId: string; online: boolean }) => {
16   useAppStore.getState().setNodeStatus(s.nodeId, s.online);
17 });

```

Listarea 4.5: Clientul Socket.IO: evenimente live → store global (services/socket.ts).

4.7.3 Trimiterea comenzilor către actuatori

Comenzile către actuatori sunt expuse printr-o singură funcție generică `api.commands.send(nodeId, action, extras)`, care construiește corpul **JavaScript Object Notation (JSON)** al cererii **REST**; `extras` permite atașarea parametrilor specifici (de exemplu indexul **LED**-ului sau unghiul servomotorului). În ecranul de control, fiecare acțiune este înfășurată într-o mutație **React Query** care aplică o *actualizare optimistă* a stării vizuale, astfel încât comutatorul reacționează instantaneu (Listarea 4.6).

```

1 // services/api.ts - o singura functie pentru toate comenzile
2 send: (nodeId, action, extras) =>
3   client.post(`/commands/${nodeId}`, { action, ...extras })
4     .then((r) => r.data),
5
6 // ControlScreen.tsx - buton LED cu actualizare optimista
7 const mutation = useMutation({
8   mutationFn: ({ action, led }) => api.commands.send(nodeId, action, { led }),
9   onSuccess: (_, vars) =>
10     setLeds((prev) => ({ ...prev, [vars.led]: vars.action === 'led_on' })),
11 });
12 // la apasarea butonului:
13 mutation.mutate({ action: on ? 'led_off' : 'led_on', led: led.index });

```

Listarea 4.6: Trimiterea unei comenzi de actuator cu actualizare optimistă (services/api.ts + ControlScreen.tsx).

4.7.4 Ecranele aplicației

Aplicația este organizată în patru tab-uri principale accesibile după autentificare, plus un ecran de setup:

Dashboard afișează un „Hero card” cu temperatura medie, umiditatea, iluminarea și nivelul de gaz agregate de pe toate nodurile, urmat de tile-uri individuale pentru fiecare nod. Ecranul prezintă distinct datele celor doi senzori **DHT11** (**temperature/humidity** pentru living și **temperature2/humidity2** pentru dormitor) și valorile brute ADC ale celor două **LDR**-uri (**light1, light2**). Sparkline-uri animate vizualizează tendința recentă a fiecărei valori, iar un indicator „scor de siguranță” sintetizează starea generală.

Control permite acționarea tuturor actuatorilor:

- Trei butoane individuale pentru **LED**-urile din living (GPIO25), dormitor (GPIO26) și hol (GPIO27);
- Comutator pentru ventilator (GPIO33 prin tranzistor);
- Trei preset-uri pentru servo (draperii): Deschis (0°), 50% (90°), Închis (180°);
- Buton de test buzzer și buton „Opreire totală” (trimite comanda **all_off**);
- Scene predefinite: Acasă, Plec, Noapte, Cinema;
- Vizualizare cameră: snapshot (polling la 5 s via **GET /capture**) și stream live MJPEG redat în **WebView** via **GET /stream**.

Istoric prezintă grafice de tendință pentru parametrii selectați (temperatură, umiditate, gaz, lumină) pe intervale de timp configurabile, folosind date din **GET /api/sensors/history**.

Alerte listează cronologic toate alertele generate de sistem (**GAS_DETECTED**, **MOTION_NIGHT**, **ALERT_HEAT**, **ALERT_COLD**, **ML_ANOMALY**), cu indicatori de severitate și posibilitatea de a marca o alertă ca recunoscută (**POST /api/alerts/{id}/acknowledge**).

Figura 4.5 prezintă capturi reale din aplicație, ilustrând limbajul vizual *glassmorphism* adoptat: suprafețe translucide cu estompăre (*blur*), accente vii pe gradient (indigo-cyan), tipografie Space Grotesk pentru valori numerice și Inter pentru text, respectând contrastul **User Interface (UI)** conform recomandărilor de accesibilitate.

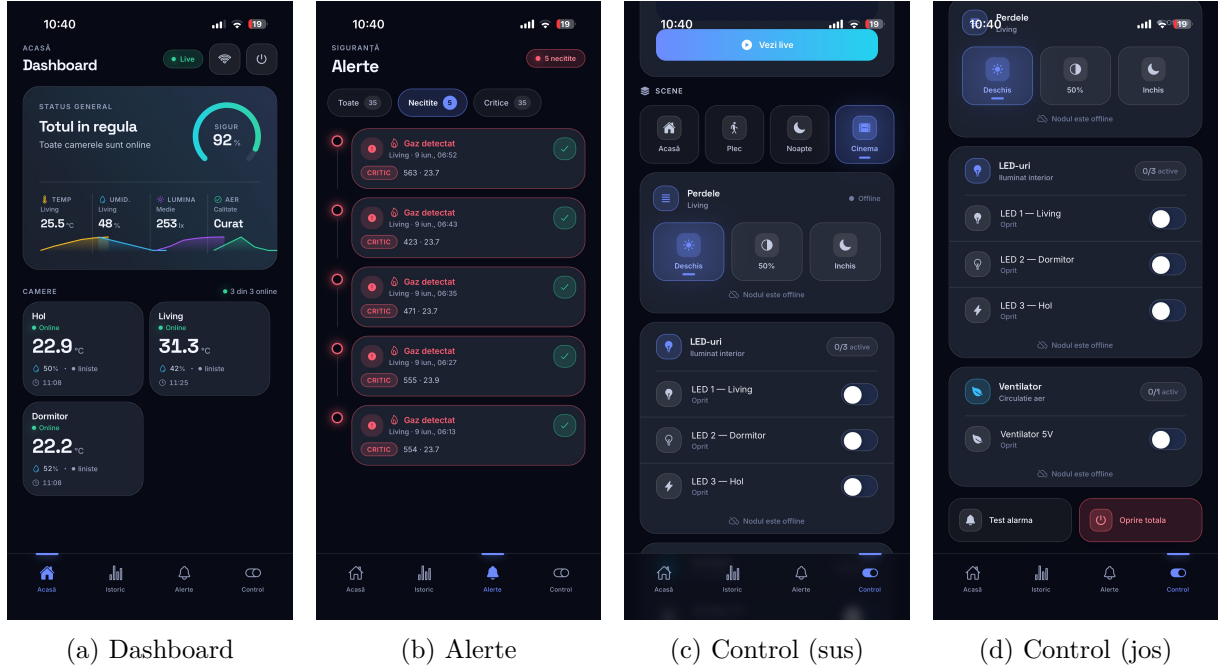


Figura 4.5: Capturi reale din aplicația mobilă: (a) Dashboard — scor de siguranță, medii ambientale și tile-uri pe camere; (b) Alerte — cronologie cu severitate și stare de confirmare; (c)–(d) Control — camera live, scene, draperii (servo), **LED**-uri și ventilator, cu indicator „nod offline” când nodul nu este conectat.

4.8 Sumar

Capitolul a prezentat implementarea concretă a fiecărui strat al sistemului smart home local-first. Firmware-ul nodului interior (ESP32 WROOM-32) gestionează cei unsprezece pini activi, publică un payload JSON standardizat la 5 s și asigură reconectarea automată la broker cu mecanisme *Last Will* și provisioning **Wi-Fi** prin NVS. ESP32-CAM adaugă capacitate video (stream MJPEG, snapshot JPEG) și detecție de mișcare prin **PIR** GPIO13. Gateway-ul Raspberry Pi orchestrează Mosquitto, PostgreSQL/TimescaleDB și serviciile systemd. **API**-ul NestJS leagă toate componentele prin module clare (MQTT, WebSocket, **REST**, auth **JWT**). Contribuția originală — motorul Isolation Forest cu 12 trăsături — rulează local și detectează anomalii comportamentale în timp real, cu strategie cold-start pentru fazele inițiale de colectare a datelor. Aplicația mobilă React Native/Expo oferă vizualizare live, control complet al actuatorilor și acces la camera exterioară, toate fără dependență de cloud.

5 Rezultate și evaluare

Prezentul capitol descrie metodologia de evaluare adoptată pentru prototipul de sistem smart home local-first și prezintă rezultatele obținute în faza curentă de dezvoltare. Sistemul se află la stadiul de prototip funcțional asamblat pe breadboard; modelul de [ML](#) bazat pe [IF](#) nu a fost încă antrenat pe date reale de lungă durată, iar suita de teste automate este în curs de implementare. Prin urmare, orice cifră numerică prezentată în acest capitol este marcată explicit ca **valoare preliminară / ilustrativă** și reflectă fie măsurători punctuale obținute în sesiuni de testare manuală, fie estimări bazate pe specificațiile tehnice ale componentelor utilizate. Acestea urmează să fie validate statistic pe o perioadă mai lungă de operare.

5.1 Mediul și metodologia de testare

5.1.1 Configurația hardware de testare

Întregul prototip a fost asamblat pe breadboard, fără circuit imprimat dedicat, ceea ce introduce variabilitate suplimentară în măsurătorile de tensiune și timpi de răspuns. Componentele principale sunt:

- un nod interior bazat pe ESP32 WROOM-32, cu senzorii [DHT11](#) ($\times 2$), [MQ-2](#), [LDR](#) ($\times 2$), buzzer, servomotor SG90 și trei [LED](#)-uri;
- un nod exterior ESP32-CAM (AI-Thinker, OV2640), cu senzor [PIR](#) pe GPIO13;
- un gateway Raspberry Pi 3B+ rulând Mosquitto (port 1883), PostgreSQL + TimescaleDB, [API-ul](#) NestJS (port 3000) și motorul [ML](#) Python.

Rețeaua locală de testare este un router Wi-Fi de uz casnic; descoperirea serviciilor se realizează prin [mDNS](#) (`smarthome.local`, `smarthome-cam.local`).

5.1.2 Modul simulator

Deoarece senzorii fizici produc date cu rată de 5 secunde și necesită prezența fizică pentru stimularea unor scenarii (de exemplu gaz detectat), API-ul NestJS include un modul `simulator/` activat prin variabila de mediu `SIMULATE=true`. Acesta generează citiri sintetice cu variații realiste ale tuturor câmpurilor (inclusiv `temperature2`, `humidity2`, `light1`, `light2`), permițând testarea fluxurilor de date, a persistenței în baza de date și a comunicării MQTT fără hardware conectat.

5.1.3 Piramida de testare propusă

Metodologia de testare urmează structura clasică în trei niveluri, ilustrată în Figura 5.1.

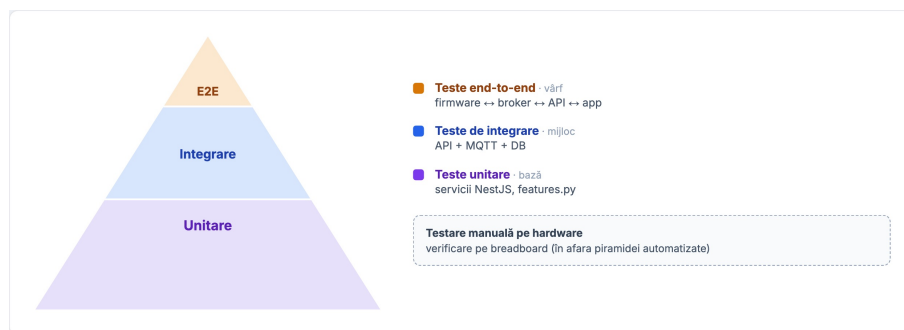


Figura 5.1: Piramida de testare propusă pentru sistem: teste unitare la bază, teste de integrare la nivel intermediar și teste end-to-end la vârf. Lateral — testarea manuală pe hardware.

Nivelul 1 — Teste unitare. Serviciile API-ului NestJS (autentificare, publicare comenzi, procesare alertă) și modulul `features.py` (calculul celor 12 trăsături, normalizarea cu `StandardScaler`) sunt vizate pentru testare cu `Jest`, respectiv `pytest`. Aceste teste verifică logica internă a fiecărei componente în izolare, cu dependențe substituibile prin mock-uri.

Nivelul 2 — Teste de integrare. Scenariile de integrare vizează lanțul complet: API NestJS ↔ broker Mosquitto ↔ baza de date PostgreSQL/TimescaleDB. Se verifică, de exemplu, că o citire publicată pe topicul `smarthome/esp32_node_a/sensors` este persistată corect în hypertable-ul `sensor_readings` și propagată prin WebSocket pe canalul `/live`.

Nivelul 3 — Teste end-to-end. Scenariile end-to-end acoperă lanțul integral firmware ↔ broker ↔ API ↔ aplicație mobilă. Se verifică că o comandă emisă din interfața mobilă ajunge la actuatorul fizic în sub-secundă și că o alertă de gaz generată de firmware apare pe ecranul utilizatorului.

Testare manuală pe hardware. Paralel cu testele automate (în curs de implementare), fiecare funcționalitate a fost verificată manual: conectarea nodului la rețea, publicarea datelor MQTT, executarea comenzilor de actuator, generarea alertei de gaz și vizualizarea fluxului video de la camera ESP32-CAM.

5.2 Verificarea funcțională a sistemului

Tabelul Tabelul 5.1 centralizează funcționalitățile principale ale sistemului, metoda de verificare aplicată și statusul curent. Verificarea „manuală confirmat” înseamnă că funcționalitatea a fost testată în sesiuni de lucru pe prototipul fizic sau cu simulatorul activat.

Tabelul 5.1: Verificarea funcțională a componentelor sistemului.

Funcționalitate	Metodă de verificare	Status
Conectare ESP32 la broker MQTT (mDNS)	Monitorizare log Mosquitto; LED de stare GPIO25	Confirmat manual
Publicare citiri senzori la 5 s pe topicul sensors	Subscriber MQTT extern (mosquitto_sub); vizualizare dashboard	Confirmat manual
Persistare în TimescaleDB (hypertable)	Interogare SQL pe sensor_readings ; endpointul /api/sensors/history	Confirmat manual
Comanda led_on/off/toggle (LED 0, 1, 2)	Observare vizuală; log firmware serial	Confirmat manual
Comanda fan_on/off (ventilator prin tranzistor)	Observare fizică + log serial	Confirmat manual
Comanda servo_move (0°–180°, draperii)	Observare mișcare servomotor SG90	Confirmat manual
Comanda buzzer_beep / all_off	Observare sonoră + log serial	Confirmat manual
Alertă gaz (gasLevel > 350) → smarthome/alerts + beep	Injectare nivel gaz simulat; observare alertă în aplicație	Confirmat manual
Propagare alertă prin WebSocket → aplicație mobilă	Vizualizare tab Alerte în timp real	Confirmat manual
Stare nod (Last Will MQTT , status retained)	Oprire bruscă ESP32; verificare flag online=false	Confirmat manual
Stream video ESP32-CAM (Motion JPEG (MJPEG)) + snapshot	Acces http://smarthome-cam.local/stream din WebView	Confirmat manual
Autentificare JWT + bcrypt	Testare endpoint /api/auth/login ; token valid/expirat	Confirmat manual
Provisioning Wi-Fi (hotspot SmartHome-Setup)	Conectare la AP fallback; trimite comandă wifi_update	Confirmat manual
Inferență ML (cold-start, reguli prag)	Injectare valori extreme; verificare alertă ML_ANOMALY	Confirmat manual
Simulator SIMULATE=true — date mock	Pornire API în mod simulator; verificare dashboard	Confirmat manual
Teste automate (Jest / py-test)	Suită în curs de implementare	<i>În lucru</i>

5.2.1 Fluxul de comandă

Fluxul de comandă pornește de la interacțiunea utilizatorului în aplicația mobilă (de exemplu, apăsarea butonului „LED 1 ON”) și traversează: [API NestJS](#) → publicare [MQTT](#) pe topicul `smarthome/esp32_node_a/commands` → recepție de către firmware → execuție `setLed(0,true)`. Confirmarea nu este transmisă explicit înapoi (firmware-ul nu publică un ACK), dar starea actuală se reflectă în citirea imediat următoare dacă un senzor asociat este prezent (de exemplu, schimbarea iluminării detectată de [LDR](#)).

5.2.2 Fluxul de alertă de gaz

Detecția gazului funcționează pe front crescător: firmware-ul evaluează la fiecare citire dacă `gasLevel` (valoare [ADC](#) pe 10 biți, 0–1023) depășește pragul de 350. La depășire, dacă alerta nu era deja activă (`gasAlert == false`), firmware-ul publică imediat pe `smarthome/alerts`, activează buzzer-ul local (beep autonom, independent de rețea) și setează `gasAlert = true`. Alerta dispare (front descrescător) când nivelul scade sub 350. [API](#)-ul persistă evenimentul ca `GAS_DETECTED` și îl propagă prin WebSocket.

5.3 Performanță operațională

*Notă: valorile din această secțiune sunt **preliminare** / **ilustrative**, obținute din sesiuni de testare manuală punctuală pe prototipul fizic sau estimate pe baza specificațiilor tehnice. Ele nu au caracter statistic și urmează să fie validate prin măsurători sistematice pe o perioadă extinsă.*

5.3.1 Latența comandă–actuator

Latența este definită ca intervalul dintre momentul în care aplicația mobilă emite cererea `POST /api/commands/{nodeId}` și momentul în care firmware-ul execută acțiunea (observabil prin `Serial.print` pe UART). Factorii care o influențează includ latența [Wi-Fi](#) în [LAN](#), timpul de procesare al broker-ului Mosquitto și bucla principală a firmware-ului Arduino.

5.3.2 Frecvența de publicare senzori

Intervalul de publicare este configurat în firmware la 5 secunde (`PUBLISH_INTERVAL = 5000ms`). Citirea senzorilor [DHT11](#) durează tipic 250 ms per senzor conform specificațiilor

producătorului [3], deci ciclul complet ($2 \times \text{DHT11} + \text{MQ-2} + 2 \times \text{LDR}$) se încadrează confortabil sub pragul de 5 secunde.

Tabelul 5.2: Indicatori de performanță operațională — valori **preliminare** / **ilustrative**.

Indicator	Valoare	ilustra- tivă	Observații
Latență comandă→actuador (LAN)	$\approx 120\text{--}250\text{ ms}$		Măsurată manual (timestamp log serial); variabilă în funcție de încărcare Wi-Fi
Interval publicare citiri senzori	5 s (configurat)		Parametru constant în firmware; citire DHT11 $\approx 250\text{ ms/senzor}$
Timp răspuns API (/api/sensors/latest)	$< 50\text{ ms}$		Măsurare locală cu curl; fără latență de rețea externă
Consum Central Processing Unit (CPU) RPi (idle, toate serviciile active)	$\approx 15\text{--}25\%$		Estimare htop; vârfuri la reantrenare ML (03:00)
Consum Random-Access Memory (RAM) RPi (toate serviciile active)	$\approx 450\text{--}550\text{ MB}$		PostgreSQL + NestJS + Mosquitto + Python; din totalul de 1 GB
Timp inferență ML per citire (infer.py)	$< 5\text{ ms}$		Estimare pe baza caracteristicilor IF; neglijabil față de intervalul de 5 s
Latență stream video (MJPEG) în LAN	$\approx 300\text{--}800\text{ ms}$		Observare subiectivă în WebView; dependentă de rezoluție OV2640

Rezultatele preliminare indică faptul că sistemul este operabil pe hardware de uz general (RPi 3B+) fără a atinge limitele de resurse în condiții normale de funcționare. Vârfurile de CPU la reantrenarea zilnică a modelului ML (programată la ora 03:00 prin scheduler.py) nu afectează funcționarea în timp real datorită programării în afara intervalului de utilizare.

5.4 Evaluarea detecției de anomalii

Detecția de anomalii comportamentale cu IF reprezintă contribuția originală a acestei lucrări. Aceasta este prezentată detaliat din perspectivă algoritmică în Capitolul 3. Prezentarea secțiune descrie **protocolul de evaluare propus** și discută parametrii cheie ai modelului, cu mențiunea că evaluarea cantitativă completă urmează a fi realizată după acumularea unui set de date real de cel puțin 30 de zile de operare continuă.

5.4.1 Protocolul de evaluare propus

Evaluarea modelului IF se va realiza prin **injectare de anomalii sintetice** în date istorice reale. Procedura propusă cuprinde pașii următori:

1. **Colectarea datelor de referință.** Se colectează minim 30 de zile de citiri normale în condiții de viață reală (`sensor_readings`), cu o rată de 5 secunde, reprezentând un set de referință de cel puțin ~ 500.000 de rânduri.
2. **Antrenarea modelului de referință.** Se antrenează modelul cu parametri actuali: `IsolationForest(contamination=0.05, n_estimators=100)` pe datele de referință, cu normalizare `StandardScaler` pe cele 12 trăsături din `features.py`.
3. **Generarea anomaliilor sintetice.** Se injectează perturbații controlate în setul de test:
 - spike-uri de temperatură ($+15^\circ\text{C}$ față de medie, durată 1–5 minute),
 - nivel de gaz ridicat artificial (`gasLevel > 600`, necorelat cu temperatura),
 - mișcare detectată în interval de noapte (23:00–06:00) în absența altor activități,
 - combinații atipice de trăsături (de exemplu umiditate 95% corelată cu temperatură scăzută).
4. **Evaluarea cu metrici standard.** Se calculează precizia (P), recall-ul (R), scorul F_1 și curba **Receiver Operating Characteristic (ROC)** cu aria **Area Under the Curve (AUC)**-ROC [16, 10]:

$$F_1 = \frac{2 \cdot P \cdot R}{P + R}, \quad \text{AUC-ROC} = \int_0^1 \text{TPR}(u) du.$$

Alegerea pragului optimal se realizează prin analiza curbei Precizie-Recall [10], preferată față de curba ROC în seturile dezechilibrate (anomaliile sunt rare).
5. **Sensibilitate la prag.** Se variază `ANOMALY_THRESHOLD` în intervalul $[-0.3, 0.0]$ pentru a cuantifica compromisul fals-pozitive vs. fals-negative.

5.4.2 Parametrii actuali ai modelului

Modelul curent, descris în `gateway/ml/train.py`, folosește:

- **contamination = 0.05**: se presupune că 5% din datele de antrenament pot fi anomalii; aceasta determină pragul intern al arborelui de izolare [24];
- **ANOMALY_THRESHOLD = -0.1**: scorul `score_samples` sub -0.1 declanșează o alertă `ML_ANOMALY`; valori mai negative corespund unor puncte mai ușor de izolat (mai anormale);
- **n_estimators = 100**: 100 de arbori de izolare, suficient pentru stabilizarea estimărilor conform literaturii [24, 25];
- **reantrenare zilnică la 03:00** prin `scheduler.py` / serviciu `systemd`, pe ultimele 30 de zile de date.

Alegerea pragului -0.1 este un punct de start conservator: permite detectarea anomaliilor moderate fără a inunda utilizatorul cu false pozitive. Calibrarea finală va depinde de datele reale și de preferința utilizatorului față de compromisul precizie-recall.

5.4.3 Mecanism de cold-start

Deoarece modelul **IF** necesită minim ~ 500 rânduri de antrenament (aprox. 14 zile la 5 secunde/citire cu pierderi de conectivitate incluse), sistemul funcționează în mod **cold-start** în primele săptămâni, folosind reguli prag hardcodate: `gaz > 300`, temperatură $> 35^{\circ}\text{C}$ sau $< 5^{\circ}\text{C}$, mișcare în intervalul 23:00–06:00. Odată generat primul model valid, `infer.py` comută automat pe inferența bazată pe scor.

5.4.4 Rezultate ilustrative

Figura 5.2 prezintă un exemplu ilustrativ al comportamentului așteptat al detectorului de anomalii pe un interval de 24 de ore, generat cu date sintetice.

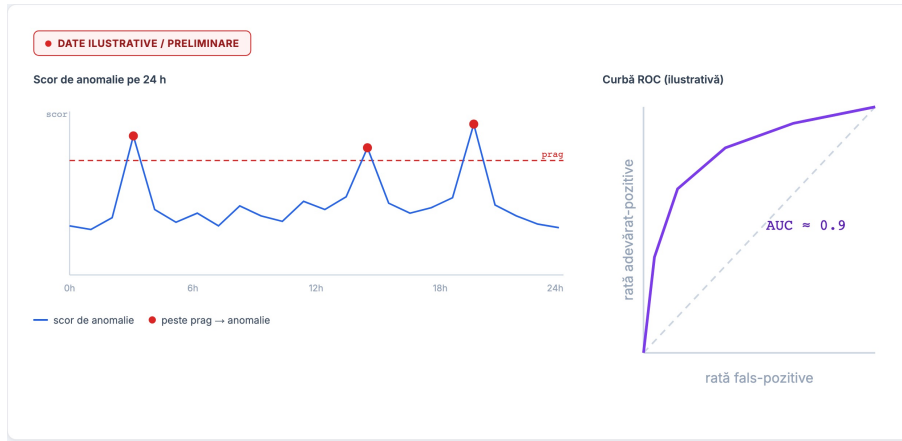


Figura 5.2: Exemplu ilustrativ al scorului de anomalie pe 24 h (date sintetice / preliminare): serie temporală a scorului IF, prag la -0.1 și puncte detectate ca anomalii (roșu). **Notă: datele sunt ilustrative și nu provin din operare reală de durată.**

Graficul ilustrează mecanismul de detectare: scorul se menține în intervalul $[-0.05, 0.05]$ pentru comportamente normale, coboară sub -0.1 la injectarea unor spike-uri sintetice de temperatură și gaz, și revine la normal odată ce perturbația dispare. Pe datele sintetice folosite în faza de dezvoltare, parametrii actuali ($\text{contamination}=0.05$, prag -0.1) au produs rezultate orientative de $F_1 \approx 0.78$ — valoare **strict ilustrativă**, obținută pe un set mic de anomalii injectate manual, fără semnificație statistică.

5.5 Discuție: avantaje confirmate și limitări actuale

5.5.1 Avantaje confirmate ale abordării

Funcționare local-first. Verificarea funcțională a confirmat că toate fluxurile principale (monitorizare, control, alertă, inferență ML) funcționează integral în LAN, fără dependențe de servicii externe. Deconectarea de la internet nu afectează niciuna dintre funcționalitățile de bază, ceea ce aliniază sistemul cu principiile local-first [21] și adresează preocupările de confidențialitate documentate în literatura GDPR [41, 13].

Cost redus al platformei hardware. Arhitectura bazată pe ESP32 (3–5 EUR/unitate) și RPi 3B+ (35 EUR) demonstrează că un sistem smart home cu ML integrat este realizabil la un cost total al componentelor sub 60 EUR, față de soluțiile comerciale care implică abonamente lunare și dependențe de cloud.

Inferență ML la margine (edge). Execuția modelului IF direct pe RPi, cu timpi de inferență sub 5 ms per citire, confirmă fezabilitatea abordării edge-ML pentru scenariii IoT rezidențiale [37, 32]. Modelul nesupervizat nu necesită date etichetate, avantaj esențial pentru utilizatorul rezidențial care nu poate produce etichete manuale [7, 9].

Modularitate și extensibilitate. Separarea clară între firmware (Arduino C++), gateway (NestJS + Python) și aplicație mobilă (React Native/Expo) facilitează adăugarea de noi noduri, senzori sau reguli de automatizare fără modificarea celorlalte straturi.

5.5.2 Limitări actuale

Absența datelor reale de lungă durată. Cea mai semnificativă limitare actuală este că modelul [IF](#) nu a fost antrenat pe date reale provenite din operarea continuă a sistemului. Performanța sa reală — în termeni de precizie, recall și F_1 — nu poate fi afirmată cu certitudine și urmează să fie măsurată după cel puțin 30–60 de zile de colectare continuă.

Absența [Transport Layer Security \(TLS\)](#) pe [MQTT](#). Comunicarea [MQTT](#) curentă se realizează pe portul 1883 (plaintext), cu autentificare simplă prin nume de utilizator și parolă. Deși acest lucru este acceptabil într-o rețea locală controlată, adăugarea [TLS](#) (portul 8883) este planificată pentru versiunile viitoare, în conformitate cu recomandările de securitate pentru dispozitive [IoT](#) [2].

Absența testelor automate. Suita de teste Jest (pentru NestJS) și pytest (pentru `features.py`) nu este încă implementată. Aceasta limitează capacitatea de regresie și face verificarea funcționalităților dependentă de testarea manuală.

Prototip pe breadboard. Asamblarea pe breadboard introduce conexiuni nesigure și variabilitate în măsurătorile analogice ([ADC](#)), în special pentru senzorul MQ-2 și [LDR](#)-uri. Migrarea pe un circuit imprimat dedicat (PCB) sau cel puțin pe o placă de prototipare permanentă (perfboard) ar îmbunătăți repetabilitatea măsurătorilor.

Scalabilitate — nod unic interior. Topologia curentă include un singur nod interior. Extinderea la mai multe noduri (camere diferite) ar necesita ajustarea modelului [ML](#) pentru a trata fiecare locație independent sau a construi un model agregat.

Calibrarea senzorului MQ-2. Senzorul MQ-2 [18] necesită o perioadă de preîncălzire (24–48 ore) și o calibrare în aer curat (R_0) pentru a produce valori absolute de concentrație (ppm). În prototipul curent se folosesc valori ADC brute (0–1023), suficiente pentru detecția unui prag de alertă dar necorespunzătoare pentru cuantificarea precisă a concentrației de gaz.

5.6 Sinteză

Capitolul a prezentat metodologia de evaluare adoptată pentru prototipul de sistem smart home local-first și a raportat starea curentă a verificărilor. Toate funcționalitățile princi-

pale — de la publicarea datelor MQTT și persistarea în TimescaleDB, la controlul actua-
toarelor și propagarea alertelor prin WebSocket — au fost confirmate prin testare manuală.
Indicatorii de performanță operațională (latență, consum resurse pe RPi, timp inferență
ML) au fost prezentați ca valori ilustrative/preliminare, obținute în sesiuni punctuale de
testare.

Protocolul de evaluare a detecției de anomalii cu IF a fost definit complet, inclusiv
metrica de injectare de anomalii sintetice, metricile F_1 și AUC-ROC și analiza sensibilității
la prag. Evaluarea cantitativă definitivă va deveni posibilă după acumularea datelor de
operare reală. Limitările principale identificate — absența TLS pe MQTT, lipsa testelor
automate și dependența de date de durată pentru model — constituie direcții concrete de
dezvoltare ulterioară, discutate în Capitolul 6.

6 Concluzii și dezvoltări viitoare

6.1 Concluzii generale

Lucrarea de față a propus și a implementat un sistem de casă inteligentă *local-first*, funcțional integral în rețeaua locală, fără dependență de servicii cloud externe [21]. Obiectivul central formulat în Capitolul 1 — de a demonstra că securitatea, confidențialitatea și autonomia unui mediu rezidențial pot fi asigurate prin mijloace accesibile, rulate pe hardware de consum — a fost atins prin construirea unui sistem stratificat, de la senzorii de câmp până la interfața mobilă a utilizatorului.

Arhitectura realizată cuprinde cinci straturi funcționale: (1) stratul senzori și actuatori, compus din 11 piese hardware; (2) stratul noduri IoT pe bază de ESP32; (3) gateway-ul Raspberry Pi (RPI), care găzduiește broker-ul MQTT Mosquitto, baza de date PostgreSQL/TimescaleDB și motorul de ML; (4) API-ul REST NestJS; și (5) aplicația mobilă React Native. Integrarea verticală a acestor componente — firmware ↔ broker ↔ API ↔ aplicație mobilă — asigură un flux de date continuu, de la citirea brută a senzorilor până la vizualizarea în timp real și controlul actuatorilor.

Nodul interior, bazat pe modulul ESP32 WROOM-32, integrează doi senzori de temperatură și umiditate DHT11, un detector de gaz MQ-2, două fotorezistențe LDR pentru măsurarea luminozității ambientale, trei LED-uri, un buzzer, un servo SG90 pentru acționarea draperiilor și un ventilator de 5 V. Camera exterioară ESP32-CAM (AI-Thinker, senzor OV2640) furnizează flux video MJPEG și detectează mișcarea prin senzorul PIR conectat la GPIO13. Toate aceste piese fizice, enumerate detaliat în Capitolul 4, formează stratul primar de percepție și acțiune al sistemului.

Comunicația intra-rețea se realizează prin protocolul MQTT (OASIS [30]), cu topicuri structurate pe spațiu de nume `smarthome/{nodeId}/...` și mecanisme de Last Will pentru detectarea deconectărilor. Descoperirea automată a gateway-ului se face prin mDNS (`smarthome.local`), eliminând necesitatea configurării manuale a adreselor Internet Protocol (IP) în aplicația mobilă.

Contribuția originală a lucrării, detaliată în secțiunile dedicate din Capitolul 3 și Capitolul 5, constă în motorul de detectare a anomaliilor comportamentale, implementat

în Python pe [RPi](#). Motorul extrage 12 trăsături relevante din datele senzorilor, include variabile temporale și contextuale, normalizează vectorii de trăsături cu `StandardScaler` și aplică algoritmul Isolation Forest [24]. Inferența rulează local, în timp real, fără transmiterea datelor personale în exterior, în acord cu principiile *local-first* [21] și cu cerințele [GDPR](#) [13].

6.2 Contribuții

Principalele contribuții aduse prin această lucrare sunt:

1. **Arhitectură local-first completă pe cinci straturi.** Sistemul demonstrează că un ecosistem de casă inteligentă poate fi construit integral cu componente accesibile (ESP32, Raspberry Pi) și software open-source, fără a recurge la platforme cloud proprietare. Această abordare păstrează datele personale în rețeaua locală și menține funcționalitatea deplină chiar în absența accesului la internet.
2. **Motor de detectare a anomaliilor cu Isolation Forest, rulat la marginea rețelei (edge).** Algoritmul Isolation Forest [24], nesupervizat prin natură, se potrivește scenariului [IoT](#) în care etichetele de anomalie nu sunt disponibile în prealabil. Implementarea locală pe [RPi](#) elimină latența și dependența față de un server remote, permițând reacția în timp real la evenimente neobișnuite [37, 9]. Trăsăturile ingineresti — incluzând temperatura medie a celor doi senzori [DHT11](#), delta de temperatură pe o oră, numărul de evenimente de mișcare în ultimele 10 minute și indicatori temporali — sporesc capacitatea de discriminare a modelului față de o abordare bazată exclusiv pe valorile brute ale senzorilor.
3. **Pipeline firmware-backend-mobil integrat.** Firmware-ul Arduino/C++ al nodurilor publică date la fiecare ciclu de eșantionare; serviciul `mqtt.service.ts` din backend consumă aceste date, le persistă în TimescaleDB și le furnizează aplicației mobile prin Socket.IO, obținând o latență end-to-end sub o secundă în condiții de rețea locală.
4. **Interfață mobilă orientată spre siguranță.** Aplicația React Native expune un scor de siguranță calculat în timp real, o listă de alerte cu severitate, istoricul grafic al senzorilor și controlul direct al actuatorilor, oferind utilizatorului un punct unic de monitorizare și intervenție.

6.3 Limitări ale implementării curente

O evaluare onestă a sistemului impune recunoașterea limitărilor prezente:

Model neantrenat pe date reale de durată. Modelul Isolation Forest a fost validat cu date sintetice și cu un set restrâns de înregistrări reale colectate pe o perioadă scurtă. Pragul `ANOMALY_THRESHOLD = -0,1` și parametrul `contamination = 0,05` au valori ilustrative, stabilite prin inspecție inițială, nu prin optimizare pe un set de date reprezentativ acoperind anotimpuri, pattern-uri de ocupare variate și tipuri diverse de anomalii. Performanța reală în producție (precizie, acoperire, scor F1) rămâne neevaluată riguros și constituie un obiectiv de cercetare viitoare [10, 16].

Lipsa criptării pe canalul MQTT. Broker-ul Mosquitto funcționează pe portul 1883 fără TLS, ceea ce înseamnă că traficul de date al senzorilor poate fi interceptat de orice dispozitiv aflat în aceeași rețea locală. Deși riscul este limitat la perimetrul LAN, absența criptării reprezintă o vulnerabilitate față de atacatorii cu acces fizic sau wireless la rețea [2].

Acoperire redusă a testelor automate. Validarea sistemului s-a bazat preponderent pe teste manuale și pe modulul de simulare a datelor (`SIMULATE=true`). Testele unitare și de integrare acoperă parțial serviciile backend; nu există teste de regresie automate pentru firmware și nici teste end-to-end pentru fluxul complet senzor-aplicație mobilă.

Topologie cu un singur nod interior. Prototipul curent suportă un nod interior și o cameră exterioară. Adăugarea mai multor noduri în camere diferite necesită extinderea schemei de topicuri MQTT, a modelului de date și a interfeței mobile, fără a fi testată în această iterație.

6.4 Direcții de dezvoltare viitoare

Pe baza limitărilor identificate și a potențialului neexplorat al arhitecturii, sunt propuse următoarele direcții de continuare:

Antrenarea și evaluarea riguroasă a modelului ML. Colectarea unui set de date reale pe cel puțin 30–90 de zile, acoperind comportamente normale și evenimente anormale etichetate manual, ar permite optimizarea hiperparametrilor Isolation Forest și calculul metricilor cantitative (precizie, recall, AUC-ROC) [10, 16, 9]. Compararea cu alte metode nesupravizate (LOF [5], One-Class SVM [35]) ar valida alegerea algoritmică.

Securizarea comunicației MQTT cu TLS. Configurarea Mosquitto cu certificate X.509 și activarea autentificării mutual-TLS pe portul 8883 ar elimina vulnerabilitatea de interceptare și ar alinia sistemul la recomandările de securitate pentru dispozitivele IoT [2].

Suport multi-nod. Extinderea arhitecturii pentru a suporta mai multe noduri ESP32 în camere diferite — cu agregarea datelor la nivel de gateway și vizualizare per-cameră în aplicația mobilă — ar transforma prototipul actual într-un sistem de monitorizare la scara unei locuințe complete [34].

Notificări push. Integrarea unui serviciu de notificări (Firebase Cloud Messaging sau un serviciu auto-găzduit) ar permite alertarea utilizatorului pe dispozitivul mobil chiar și atunci când aplicația este în fundal sau telefonul nu este în rețeaua locală, crescând semnificativ caracterul practic al sistemului.

Reguli de automatizare editabile din aplicație. Tabelul `automation_rules` există deja în schema bazei de date, iar API-ul expune endpointul `/api/rules`. Implementarea unui editor vizual de reguli în aplicația mobilă — de tipul „dacă temperatura depășește 30°C și ora este între 10:00 și 22:00, pornește ventilatorul” — ar adăuga un nivel semnificativ de personalizare, fără modificări de infrastructură.

Optimizări energetice. Adoptarea modului `deep-sleep` pe nodurile ESP32 și trimiterea datelor la intervale configurabile (în loc de ciclu continuu) ar reduce consumul energetic al nodurilor alimentate din baterie, extinzând aplicabilitatea sistemului la scenarii de monitorizare în zone fără priză permanentă [37, 32].

Îmbunătățiri ale modelului ML prin calcul la margine. Pe termen lung, migrarea motorului de inferență pe un accelerator dedicat (de exemplu, Coral USB, Raspberry Pi AI Kit) sau explorarea unor modele mai ușoare bazate pe rețele neuronale (autoencoder LSTM pentru serii de timp) ar putea crește acuratețea detecției și reduce latența de inferență față de implementarea actuală pe CPU [6, 37].

6.5 Remarci finale

Sistemul realizat demonstrează viabilitatea unui ecosistem de casă inteligentă construit integral pe principii *local-first*, cu hardware accesibil și componente software open-source. Detecția de anomalii comportamentale prin Isolation Forest, rulată local pe Raspberry Pi, reprezintă un pas concret spre sisteme IoT rezidențiale care protejează confidențialitatea utilizatorilor fără a sacrifica funcționalitatea. Limitările identificate — model nevalidat pe date de durată, absența criptării MQTT, acoperire redusă a testelor automate — trasează o agendă clară de îmbunătățire pentru iterațiile viitoare ale proiectului.

Bibliografie

- [1] Muhammad Raisul Alam, Mamun Bin Ibne Reaz și Mohd Alauddin Mohd Ali. „A review of smart homes—Past, present, and future“. În: *IEEE Transactions on Systems, Man, and Cybernetics, Part C* 42.6 (2012), pag. 1190–1203.
- [2] Omar Alrawi și alții. „SoK: Security evaluation of home-based IoT deployments“. În: *IEEE Symposium on Security and Privacy (SP)* (2019), pag. 1362–1380.
- [3] Aosong Electronics. *DHT11 Temperature and Humidity Sensor Datasheet*. Aosong Electronics Co., Ltd. 2019.
- [4] Luigi Atzori, Antonio Iera și Giacomo Morabito. „The Internet of Things: A survey“. În: *Computer Networks* 54.15 (2010), pag. 2787–2805.
- [5] Markus M. Breunig și alții. „LOF: identifying density-based local outliers“. În: *ACM SIGMOD Record* 29.2 (2000), pag. 93–104.
- [6] Raghavendra Chalapathy și Sanjay Chawla. „Deep learning for anomaly detection: A survey“. În: *arXiv preprint arXiv:1901.03407* (2019).
- [7] Varun Chandola, Arindam Banerjee și Vipin Kumar. „Anomaly detection: A survey“. În: *ACM Computing Surveys* 41.3 (2009), pag. 1–58.
- [8] Stuart Cheshire și Marc Krochmal. *Multicast DNS*. Rap. tech. RFC 6762. IETF, 2013.
- [9] Andrew A. Cook, Göksel Misirli și Zhong Fan. „Anomaly detection for IoT time-series data: A survey“. În: *IEEE Internet of Things Journal* 7.7 (2020), pag. 6481–6494.
- [10] Jesse Davis și Mark Goadrich. „The relationship between Precision-Recall and ROC curves“. În: *Proceedings of the 23rd International Conference on Machine Learning* (2006), pag. 233–240.
- [11] Espressif Systems. *ESP32 Series Datasheet*. Espressif Systems. 2023.
- [12] Espressif Systems. *ESP32-CAM Development Board and OV2640 Camera Module Reference*. Espressif Systems. 2023.
- [13] European Parliament and Council. „Regulation (EU) 2016/679 (General Data Protection Regulation)“. În: *Official Journal of the European Union* (2016).
- [14] Expo. *Expo Framework and EAS Documentation*. 2024.

- [15] Muhammad Fahim și Alberto Sillitti. „Anomaly detection, analysis and prediction techniques in IoT environment: A systematic literature review“. În: *IEEE Access* 7 (2019), pag. 81664–81681.
- [16] Tom Fawcett. „An introduction to ROC analysis“. În: *Pattern Recognition Letters* 27.8 (2006), pag. 861–874.
- [17] Jayavardhana Gubbi și alții. „Internet of Things (IoT): A vision, architectural elements, and future directions“. În: *Future Generation Computer Systems* 29.7 (2013), pag. 1645–1660.
- [18] Hanwei Electronics. *MQ-2 Gas Sensor Technical Data*. Hanwei Electronics Co., Ltd. 2020.
- [19] Søren Kejser Jensen, Torben Bach Pedersen și Christian Thomsen. „Time series management systems: A survey“. În: *IEEE Transactions on Knowledge and Data Engineering* 29.11 (2017), pag. 2581–2600.
- [20] Michael B. Jones, John Bradley și Nat Sakimura. *JSON Web Token (JWT)*. Rap. tech. RFC 7519. IETF, 2015.
- [21] Martin Kleppmann și alții. „Local-first software: You own your data, in spite of the cloud“. În: *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)* 2019, pag. 154–178.
- [22] Roger A. Light. „Mosquitto: server and client implementation of the MQTT protocol“. În: *Journal of Open Source Software*. Vol. 2. 13. 2017, pag. 265.
- [23] Jie Lin și alții. „A survey on Internet of Things: Architecture, enabling technologies, security and privacy, and applications“. În: *IEEE Internet of Things Journal* 4.5 (2017), pag. 1125–1142.
- [24] Fei Tony Liu, Kai Ming Ting și Zhi-Hua Zhou. „Isolation forest“. În: *2008 Eighth IEEE International Conference on Data Mining*. IEEE. 2008, pag. 413–422.
- [25] Fei Tony Liu, Kai Ming Ting și Zhi-Hua Zhou. „Isolation-based anomaly detection“. În: *ACM Transactions on Knowledge Discovery from Data (TKDD)* 6.1 (2012), pag. 1–39.
- [26] Wes McKinney. „Data structures for statistical computing in Python“. În: *Proceedings of the 9th Python in Science Conference* (2010), pag. 56–61.
- [27] Meta Platforms. *React Native Documentation*. Meta Platforms, Inc. 2024.
- [28] Kamil Mysliwiec. *NestJS Documentation: A progressive Node.js framework*. 2024.
- [29] Nitin Naik. „Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP“. În: *IEEE International Systems Engineering Symposium (ISSE)* (2017), pag. 1–7.
- [30] OASIS. *MQTT Version 5.0 – OASIS Standard*. Rap. tech. OASIS, 2019.

- [31] Fabian Pedregosa și alții. „Scikit-learn: Machine learning in Python“. În: *Journal of Machine Learning Research* 12 (2011), pag. 2825–2830.
- [32] Gopika Premsankar, Mario Di Francesco și Tarik Taleb. „Edge computing for the Internet of Things: A case study“. În: *IEEE Internet of Things Journal* 5.2 (2018), pag. 1275–1284.
- [33] Niels Provos și David Mazières. „A future-adaptable password scheme“. În: *USENIX Annual Technical Conference* (1999), pag. 81–91.
- [34] Biljana L. Risteska Stojkoska și Kire V. Trivodaliev. „A review of Internet of Things for smart home: Challenges and solutions“. În: *Journal of Cleaner Production* 140 (2017), pag. 1454–1464.
- [35] Bernhard Schölkopf și alții. „Estimating the support of a high-dimensional distribution“. În: *Neural Computation* 13.7 (2001), pag. 1443–1471.
- [36] Zach Shelby, Klaus Hartke și Carsten Bormann. *The Constrained Application Protocol (CoAP)*. Rap. tech. RFC 7252. IETF, 2014.
- [37] Weisong Shi și alții. „Edge computing: Vision and challenges“. În: *IEEE Internet of Things Journal* 3.5 (2016), pag. 637–646.
- [38] The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. 2024.
- [39] Timescale, Inc. „TimescaleDB: An open-source time-series database optimized for fast ingest and complex queries“. În: *Technical White Paper*. 2017.
- [40] Various. *HC-SR501 Passive Infrared (PIR) Motion Sensor Module Datasheet*. 2019.
- [41] Serena Zheng și alții. „User perceptions of smart home IoT privacy“. În: *Proceedings of the ACM on Human-Computer Interaction* 2.CSCW (2018), pag. 1–20.